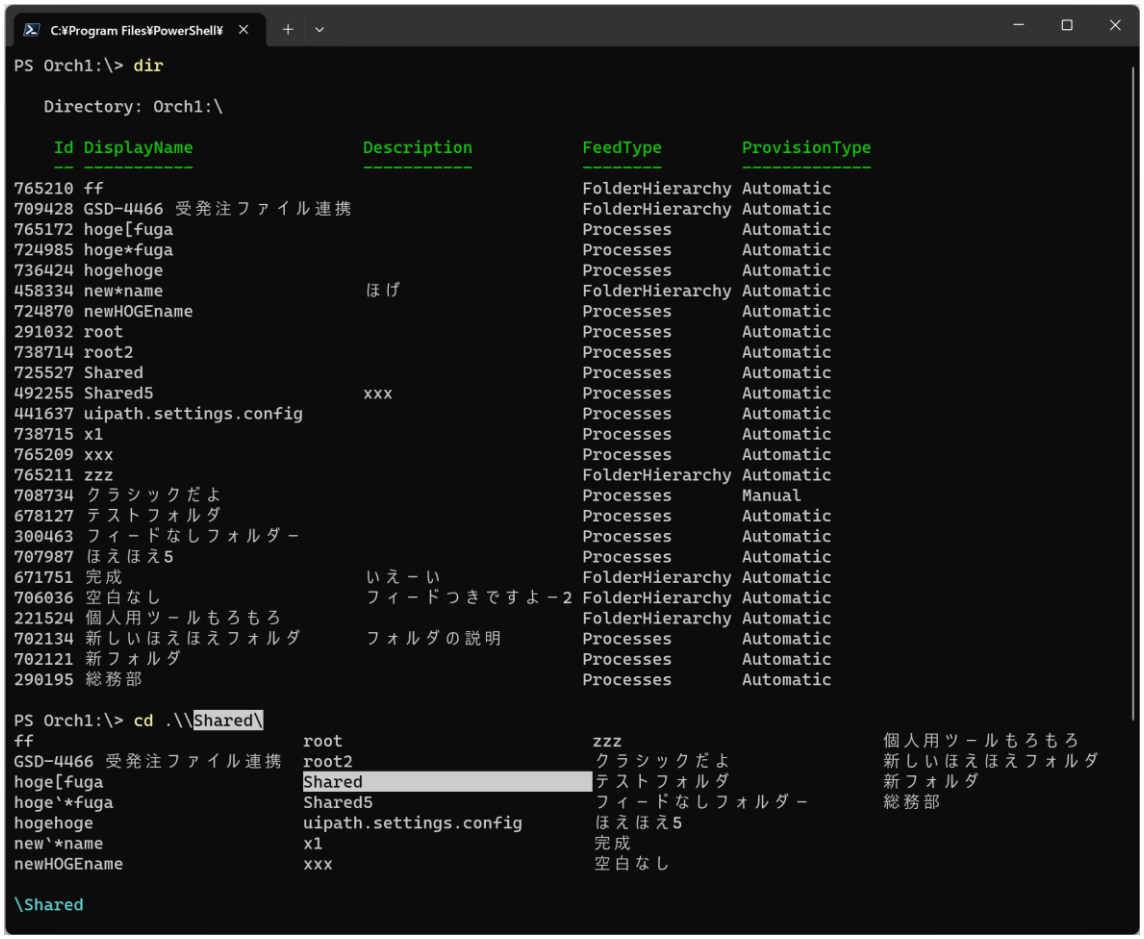
UiPathOrch Manual

Contents

Overview

UiPath Orchestrator is part of UiPath's robotic process automation (RPA) platform that allows you to manage, monitor, and execute automation tasks from a web screen. Although the operation on the web screen is convenient, it is difficult to automate the operation on Orchestrator itself or operate it with scripts. UiPathOrch solves this challenge and is implemented as a module in PowerShell.



Accessing the Latest Documentation

This document is included in the installation directory of UiPathOrch. UiPathOrch is frequently updated. Please use the following command to install the latest version as appropriate.

```
PS> Update-Module UiPathOrch
```

To refer to the latest version of this document, please open the installation directory of

UiPathOrch. After importing UiPathOrch, you can navigate to the installation directory using the following commands:

```
PS> Set-OrchLocation
PS> ii .
```

Note: ii is an alias for *Invoke-Item*. The period (.) represents the current folder.

Features of UiPathOrch

- Mounts the Orchestrator tenant as a PSDrive (PowerShell Drive) so that you can manipulate the Orchestrator folder with familiar commands such as *dir*, *cd*, and *mkdir*.
- You can mount multiple tenants, each as a separate drive, at the same time. Each tenant can be connected with a different account.
- It supports both Automation Cloud Orchestrator and on-premise Orchestrator. Each of them can be connected as a separate drive at the same time.
  (Tested with on-premise Orchestrator 2022.10)
- From the command line, you can easily perform many operations.
- You can specify multiple entities in bulk with comma-separated parameter values and wildcards.
- You can also work with multiple tenants and folders at once by specifying them with the -Path parameter, or by specifying the -Recurse parameter.
- By creating scripts and combining them with standard cmdlets, more advanced automation can be achieved.
- The parameter completion function automatically displays the correct parameter values on the console. For example, you can enter a folder name, a user name, or a machine name from several possible values. You can also view the current asset value, so edit and update it easily.
- It does not require the UiPath Assistant to be installed and licensed.
- It works very fast. If possible, it automatically sends multiple requests to Orchestrator at the same time, and immediately prints each response back to the console.
- Once the entity information is retrieved, it is cached in memory. If the user repeats the same operation, the cache is used to return the result immediately, ensuring that UiPathOrch never makes the same API call to the Orchestrator twice. (However, for some entities such as logs, the latest status is queried each time without caching.) Additionally, you have the option to instruct the system to clear the cache.

A similar tool is Orchestrator Manager. UiPathOrch is not a complete replacement for Orchestrator Manager.

Operations UiPathOrch supports

Folders

- Browse, new, remove, move and copy modern folders
- Browse, remove, and copy classic folders (copying a classic folder migrates it to a modern folder)
- Show folders' usage information
- Open folders in the browser

Note that the operation of copying a folder does not copy the contents of the folder.

Library Packages

- Get Library Packages' Names and Versions in the tenant's Library feed
- Remove Library Packages
- Copy library packages directly between tenants
- Upload Library Packages
- Download Library Packages

Process Packages

- Get the Tenant's Package Feed and the Process Package Names and Versions in the Folder with the Feeds
- Remove Process Packages
- Directly copy process packages to any feed within the same tenant
- Directly copy process packages to any feed in a different tenant
- Upload Process Packages
- Download Process Packages
- Update the process packages assigned to a folder to the latest version
- Update Process Packages Assigned to a folder to a Specific Version
- Rollback a Process Package Assigned to a folder to a Previous Version

Processes

- Get, Add, Update, Remove, and Copy processes
- Update and reset process versions

Personal Workspaces

- Displaying one's own personal workspace folder with the dir command
- Get and remove personal workspace folders
- Displaying other people's personal workspace folders that are being explored with the dir command
- Operating on personal workspaces displayed by the dir command in the same way as other folders (such as uploading/downloading packages or getting/updating assets)

Licenses

- Get Named User Licenses
- Get Runtime Licenses
- Enable Runtime Licenses
- Disable Runtime Licenses

Sessions

- Get User Sessions
- Get Machine Sessions

Jobs

- Open Job details in the browser
- Remove Jobs in Each Folders
- Start and Stop/Kill Jobs
- Get logs for each Job

Job Recordings

- Get Jobs that have Recordings (video) attached.
- Get Jobs that have Recordings (screenshots) attached.

- Save Job Recordings (screenshots) to a file.
- Remove recordings (screenshots) from Jobs.

Machines

- Add, Get, and Remove Machines
- Copy tenant machines across tenants
- Add (assign) and Remove (unassign) machines from folders
- Copy (assign) machines assigned to a folder to other folders within the same tenant or across different tenants

Roles

- Get and Remove Roles
- Get what permissions a role includes
- Copy roles across tenants

Users

- Get and Remove Tenant Users
- Add Roles to Tenant Users
- Remove Roles from Tenant Users
- Copy Tenant Users between Tenants
- Add Robot Accounts
- Add (assign) and Remove (unassign) users to/from folders
- Copy (assign) users assigned to a folder to other folders within the same tenant or across different tenants
- Add and Remove roles to/from users who are assigned to folder
- Get the current user connected to the tenant in the current PowerShell session
  (Drives connected with non-confidential apps only)
- Update the current user's password for the Unattended Robot
  (Drives connected with non-confidential apps only)

Assets

- Get, Add, Update, and Remove Assets
- Get, Add, Update, and Remove credential assets (credential store can be specified)

- Add, Update, and remove per-user and per-machine assets/credential assets
- Copy assets/credential assets to other folders within the same tenant or across different tenants
- Importing/exporting CSV file
- Get and Remove Credential Stores

Queues

- Get, Add, Update, Remove, and Copy Queues
- Get Queue Items
- Upload CSV files and bulk add Queue Items

Triggers

- Get and Remove Triggers in folders
- Enabling and disabling Triggers
- Copy Triggers

API Triggers

- Get and Remove API Triggers in folders
- Enabling and disabling API Triggers
- Copy API Triggers

Tests

- Get and Remove Test Cases
- Get, Remove, and Start Test Sets
- Get and Stop Test Set Executions
- Get, Copy, and Remove Test Set Schedules
- Get, Copy, and Remove Test Data Queues
- Get Test Data Queue Items

Calendars

- Get Calendars
- Remove Calendars
- Copy Calendars to other tenants

Storage Buckets

- Get Storage Buckets
- Remove Storage Buckets
- Copy Storage Buckets

Webhooks

- Get Webhooks
- Copy Webhooks
- Remove Webhooks
- Enable/Disable Webhooks

Stats

- Get Job Stats
- Get License Stats

Misc

- Get Robots
- Get Audit logs
- Get Alerts

Platform Management

- Get Users
- Remove Users
- Get Robot Accounts
- Create and Update Robot Accounts
- Copy Robot Accounts
- Remove Robot Accounts
- Get Groups
- Remove Groups
- Add members to Groups
- Remove members to Groups
- Get External API Resources

- Get External Apps

Document Understanding

- Get Projects
- Get Document Types

Operating UiPathOrch

- Clear UiPathOrch in-memory cache
- Opening the UiPathOrch configuration file in text editor
- List UiPathOrch drives
- Move to the UiPathOrch installation directory

Installing UiPathOrch

Considerations

- UiPathOrch requires PowerShell 7.x to install and run.
- You do not need administrator privileges to install UiPathOrch. This will be installed for the current user.
- Prior to installing UiPathOrch, change the execution policy for the current user to *bypass*.
- UiPathOrch is registered in the PowerShell Gallery, so you can easily install it if your PC is connected to the net.
  https://www.powershellgallery.com/packages/UiPathOrch/
- UiPathOrch supports both OAuth confidential and non-confidential apps for connecting to Orchestrator. Additionally, for Orchestrator versions 20.10 and earlier, connections using username/password are also supported.
- We generally recommend that you use non-confidential apps because they are more secure. Connecting with a confidential app may be necessary, for example, if you need to run a script on an unattended PC. In the following steps, we'll show you how to set up non-confidential apps.

Connection	App Secret (password) in the configuration file	User logs in with browser	UiPathOrch To grant

OAuth Confidential App	Required (Be careful not to leak to the outside)	Not required	App Scope
OAuth Non-Confidential App	Not required (safe)	Required	User Scope
Username/Password	Required (Be careful not to leak to the outside)	Not required	N/A

Installation and Setup Instructions

1. **Installing PowerShell 7.x**
2. **Installing UiPathOrch**
3. **Registering OAuth External Applications**
4. **Setting up UiPathOrch**

1. Installing PowerShell 7.x

1-1. Type [Win+R]cmd[Enter] to start the command prompt.

1-2. Enter the following command to install PowerShell.

winget install --id Microsoft.Powershell --source winget

1-3. After the installation is complete, close the command prompt window.

2. Installing UiPathOrch

2-1. Type [Win+R]pwsh[Enter] to start the PS console.

2-2. Enter the following command to install the UiPathOrch module.

Install-Module UiPathOrch

3. Registering OAuth External Application

This article walks you through the steps for creating an external application in Automation Cloud. For information about creating external applications in the on-premises version, see the user guide.

■ Manage Automation Cloud external applications

https://docs.uipath.com/ja/automation-cloud/automation-cloud/latest/admin-guide/managing-external-applications

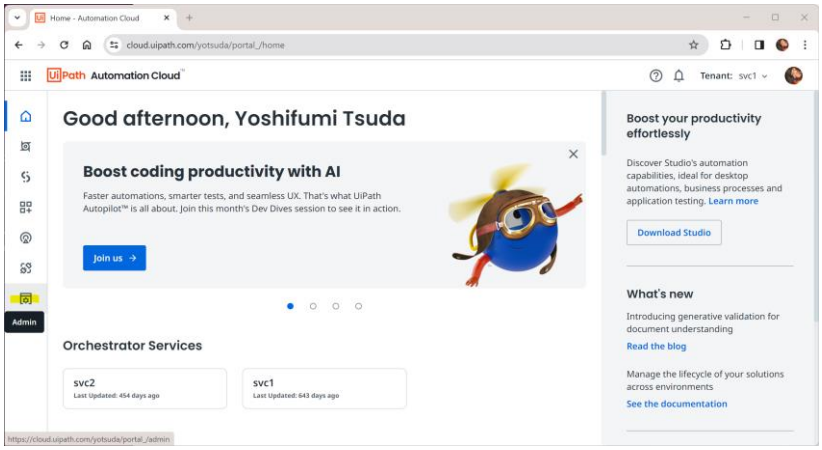
■ Manage on-premise external applications

https://docs.uipath.com/ja/orchestrator/standalone/2023.10/user-guide/managing-

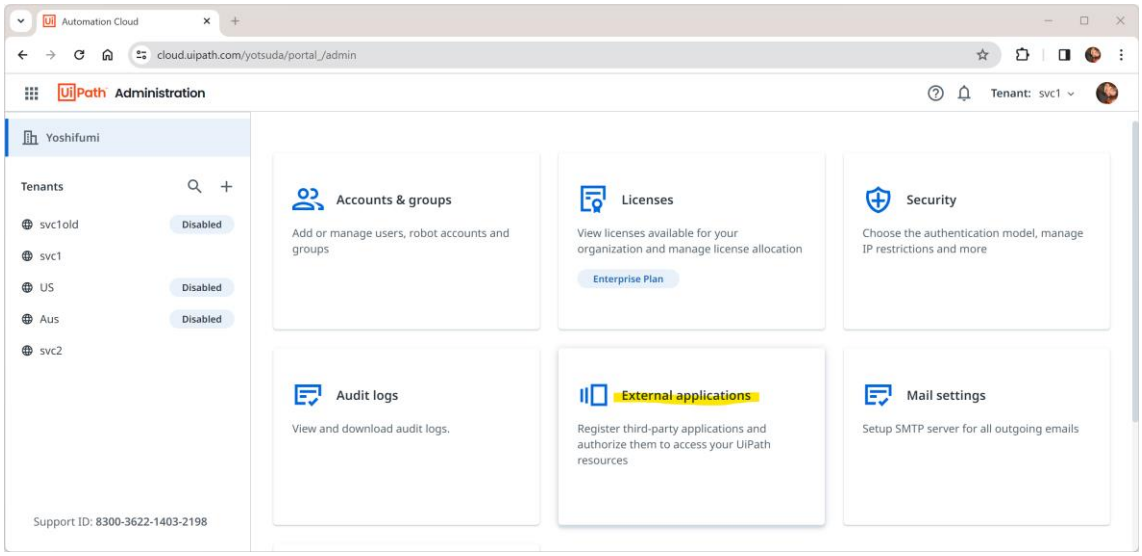
external-applications

Open the Orchestrator you want to connect to in a browser and create an OAuth external application. The App ID will be entered later in the UiPathOrch configuration file.

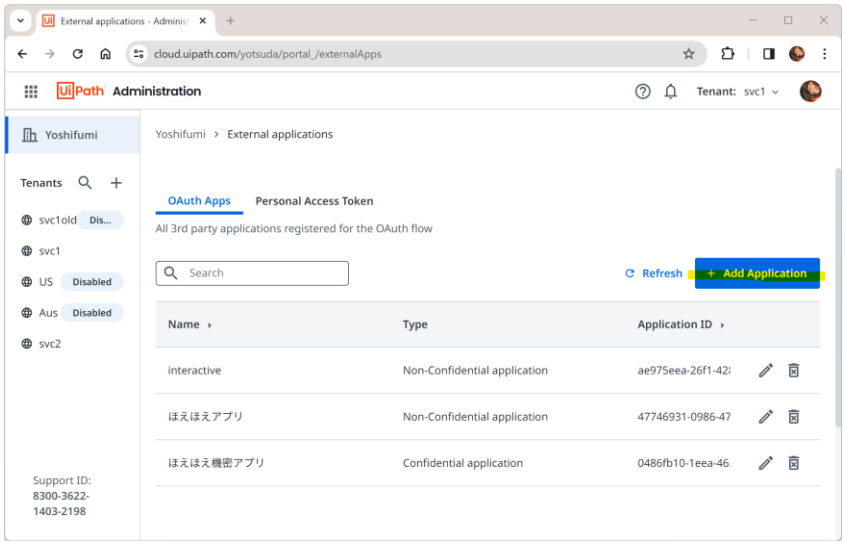
3-1. Open the Orchestrator management screen.



3-2. Open External applications.



3-3. Click [Add application].



3-4. Give an arbitrary name to [Application name].

3-5. Check [Non-confidential Application].

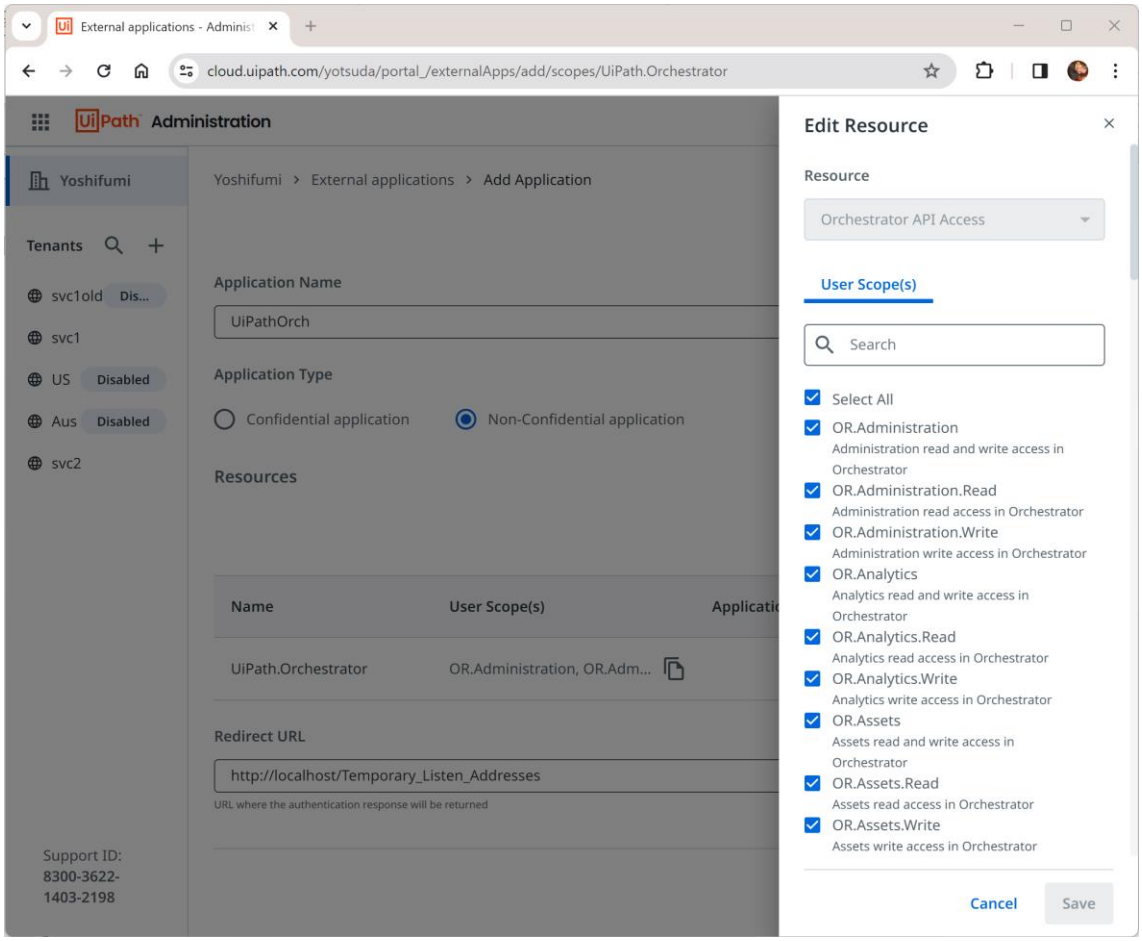
3-6. Click [Add scope], select all the [User scopes] in the resource [Orchestrator API Access], and click [Save].

3-7. Click [Add scope], select all the [User scopes] in the resource [Platform Management API Access], and click [Save].

3-8. In [Redirect URL], enter the following.

http://localhost/Temporary_Listen_Addresses

3-9. Click on the [Add] button. Then, copy the displayed application ID to the clipboard.



4. Setting up UiPathOrch

4-1. On the PS console, enter the following command to import the UiPathOrch module:

Import-Module UiPathOrch

4-2. For the first time only, a configuration file is automatically created and opened with Notepad. To manually open the configuration file, run *Edit-OrchConfig* on the PS console.

4-3. Since the part where the Name is Orch1 is an example of the setting of a non-confidential application, rewrite the setting of this part.

- ✧ Name: The drive names of the PSDrive. You can use Orch1 as is, but it is recommended to use the same name as the tenant name to be mounted.
- ✧ Root: Enter the URL of the Orchestrator you want to mount.
- ✧ AppId: Enter the app ID you paid out in step 3.

You don't need to modify RedirectUrl, HttpListener, or Scope.

4-3. Save and close the configuration file.

4-4. Close the PS console.

4-5. Type [Win+R]pwsh[Enter] to reopen the PS console.

4-6. On the PS console, enter the following command to import the UiPathOrch module:

Import-Module UiPathOrch

4-7. On the PS console, enter the following command and verify that the UiPathOrch PSDrive is mounted:

Get-PSDrive

UiPathOrch is now ready to use. From now on, all you have to do is launch the PS console and run *Import-Module UiPathOrch* to get UiPathOrch right away.

If you receive a warning stating that PSReadLine is disabled upon starting the PS console, functionalities like Tab or Ctrl+Space completion will not be operational. To resolve this issue, execute *Import-Module PSReadLine*. This will enable the completion features.

If you want your PS console to be able to use UiPathOrch automatically when it starts, fill in the following two lines in your PS profile and save it:

```
Import-Module PSReadLine,UiPathOrch
```

To open your PS profile in Notepad, run *notepad $profile* on your PS console.

Upgrading UiPathOrch

New versions of UiPathOrch will be released from time to time. To install the new version of UiPathOrch, do the following on your PS console:

```
PS> Update-Module UiPathOrch
```

This command does not remove the existing configuration file, but preserves it.

Setting Permissions

By default, the available operations are restricted for security reasons. Please grant the necessary permissions before use. For example, the following command retrieves all assets within the tenant, but it will not return results due to a permission error in the default state. (The OR.Assets.Read scope is required.)

```
PS Orch1:¥> Get-OrchAsset -Recurse
```

Please avoid granting unnecessary permissions. For instance, if you grant the OR.Folders.Write permission, the following cmdlet can be executed, which will immediately remove all folders within the tenant without confirmation. Be aware that operations that modify or remove entities cannot be undone. If you do not grant such permissions, even if such a command is mistakenly executed, it will not be processed.

```
PS Orch1:¥> rmdir *
```

Setting Scopes

To use each cmdlet, you need to add OAuth scopes. Grant scopes to external applications in the Orchestrator's management page, and request the scopes in the UiPathOrch configuration file. If you request scopes that are not permitted, the connection will fail.

- For non-confidential OAuth applications, please allow user scopes.
- For confidential OAuth applications, please allow app scopes.
- You can check the scopes required for each cmdlet execution using the following command:

```
PS> Get-Help <cmdlet name>
```

- Scopes are in one of the following three formats:
  - OR.XXX
  - OR.XXX.Read
  - OR.XXX.Write
- OR.XXX combines both OR.XXX.Read and OR.XXX.Write. If you specify OR.XXX, you do not need to specify OR.XXX.Read/OR.XXX.Write.

Adding User Accounts to Folders

To operate on entities within each folder, your account must be added to that folder with the appropriate role. To add a user to all folders directly under the root using this module, use the following command:

```
PS Orch1:¥> Add-OrchFolderUser -Path * DirectoryUser <your name> 'Folder Administrator' -Verbose
```

The OR.Folders.Write permission is required to execute the above command. Additionally, the user with administrative privileges must connect via a non-confidential application. In the

example above, the 'Folder Administrator' role is granted, but please select the appropriate role as needed.
When entering <your name> and the role name, you can press [Ctrl+Space] to auto-complete and make entry easier. Auto-completion is also available when entering cmdlet names (e.g., Add-OrchFolderUser) and parameter names (e.g., -Path). To auto-complete a parameter name, type - (minus) and then press [Ctrl+Space].
You can check the username currently connected via a non-confidential application with the following cmdlet. Please add this user to each folder. The OR.Users.Read scope is required to run this cmdlet.

```
PS Orch1:¥> Get-OrchCurrentUser
```

Quick Start

Once you've set up UiPathOrch, launch the PS console, run *Import-Module UiPathOrch*, and then try running the following command:

- *Get-PSDrive*

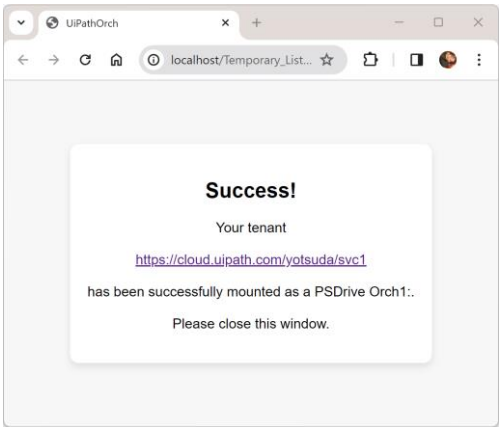
You will see the PSDrive mounted in the current PS session.

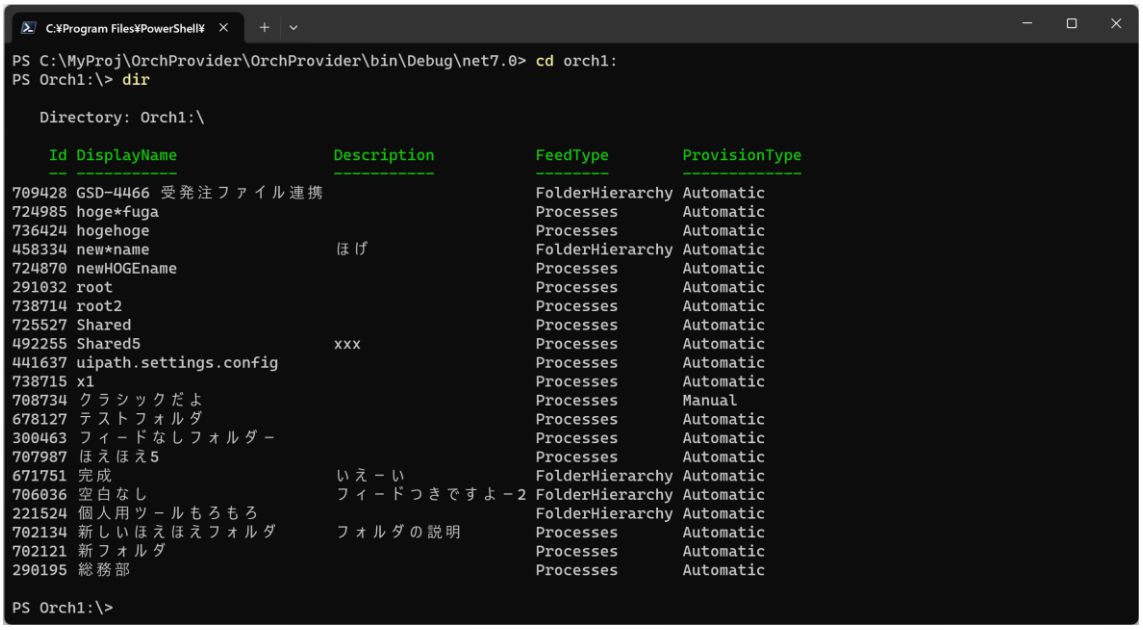
- *cd Orch1:*

Navigate to the mounted Orch drive. If you have changed the drive name in the configuration file, specify that name. (You can check the drive names with *Get-PSDrive*) Add a colon at the end of the drive name. Note that the drive name and file name are not case-sensitive.

- *dir*

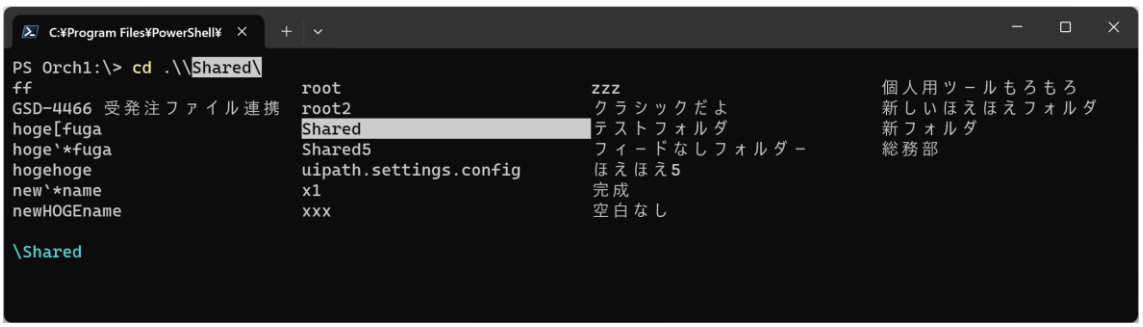
Your browser opens and you are prompted to log in to Orchestrator. Once logged in, the folders for this tenant will be listed on the PS console. Note that if you connect to Orchestrator with a non-confidential app, UiPathOrch operates under your privileges. Remember that with UiPathOrch, you cannot access folders to which you do not have permissions.





- *cd [Ctrl+Space]*

Try auto-completion feature. The destination folders are listed, and you can select and enter a folder name from them.



- *dir s**

Only folders that begin with s are listed.

- *dir a[Ctrl+Space]*

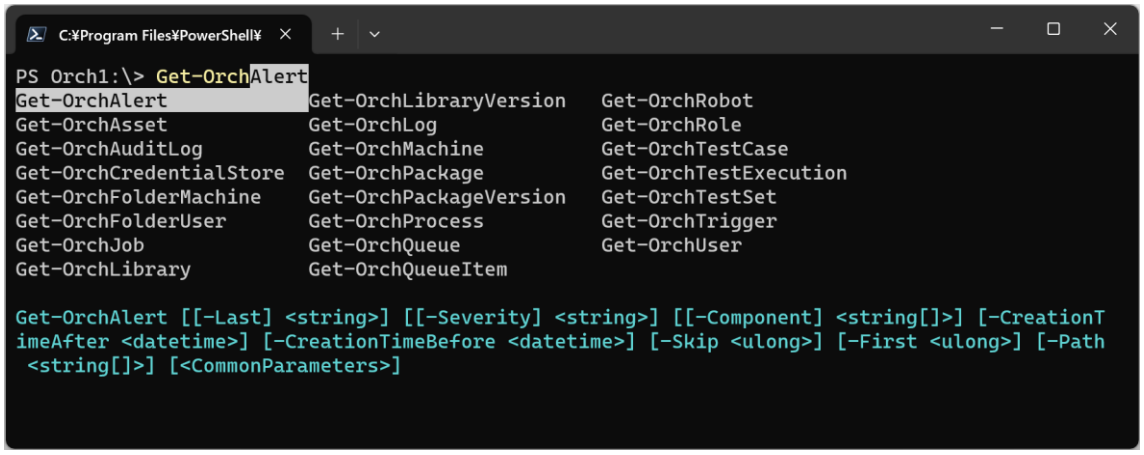
If you enter a wildcard pattern and then press [Ctrl+Space], only folders that match the pattern will be listed. In the above example, only folders starting with A are listed.

- *Get-Command -Module UiPathOrch*

Show all the commands included in the UiPathOrch module.

- *Get-Orch[Ctrl+Space]*

Try completing the cmdlet name as well. The noun part of a UiPathOrch cmdlet name always begins with Orch. In addition to Get-Orch, you can also try Add-Orch, Set-Orch, Remove-Orch, etc.



- *Get-OrchAsset -Recurse*

List the assets of all subfolders.

- *Get-OrchAsset -Recurse -ExpandUserValue*

Lists the values per user machine for assets in all subfolders.

- *Get-OrchUser a**

Show all users whose name starts with a. You can also try pressing [Ctrl+Space] instead of a*.

- *Get-OrchUser a*,y**

Display all users whose names start with a and those whose names start with y. You should also try to use [Ctrl+Space] after commas.

So far, we've shown you some samples of cmdlets that are easy to run. For more information on how to use these other cmdlets, see the cmdlet reference in this document.

About PowerShell

UiPathOrch is designed to follow the etiquette of standard PowerShell cmdlets, so users familiar with PowerShell can get started quickly. On the other hand, if you are new to PowerShell, it may be confusing. In this section, the common PowerShell operations necessary for using the UiPathOrch module are covered.

About cmdlets

Commands that can be used in the PowerShell console are called cmdlets. The name of the cmdlet is always in the form ***verb-noun***. For example, some of the standard cmdlets that you use most often include:

- *Set-Location*
- *Get-ChildItem*
- *Select-Item*
- *Format-List*
- *ConvertTo-Json*
- *Import-Csv*

There are many other cmdlets. Cmdlet names and parameter names are not case-sensitive.
Hint: The UiPathOrch module also contains many cmdlets, the noun part of which always starts with Orch. How to use them is described below.

Cmdlet aliases

Some cmdlets have aliases. For example, the alias for *Set-Location* is *cd*, and the alias *for Get-ChildItem* is *dir*. These are the names of commands that do the same thing in the Windows command prompt, Unix shell, etc., so users who are familiar with them can immediately use PowerShell. You can check the list of aliases with the *Get-Alias* cmdlet. To define a new *alias, use the Set-Alias* cmdlet.

Cmdlet parameters

Many cmdlets have parameters. A parameter always has a name. Like cmdlet names, parameter names are not case-sensitive. For example, the *Set-Location* cmdlet has -Path parameter. The parameter name is preceded by a - (hyphen). Specify the value of the parameter immediately after it. For example, to specify the value c:¥temp for the -Path parameter:

Set-Location -Path c:¥temp

Note that there are some parameters for which you do not need to specify a value. This is called a switch parameter. For example, -Recurse, -WhatIf, -Confirm, etc., described below, are switch parameters.

Positional Parameters

Some parameters have a defined position. This is called a positional parameter. When you specify a value for a positional parameter, you do not need to specify its name. For example, -Path in the cmdlet *Set-Location* is a positional parameter, and its location is zero (immediately after the cmdlet name). As such, you can also do the following:

Set-Location c:¥temp

Completion features

When you type cmdlet and parameter names, you can take advantage of completion. Completion is also available for some parameter values when entering. The completion function displays suggestions for reasonable values by pressing the Tab or Ctrl+Space keys. UiPathOrch cmdlets have completion for all parameter values.

Note that if you get a warning that PSReadLine is disabled immediately after starting the PS console, the parameter completion function will not work. To make this work, run *Import-Module PSReadLine*. You can find out more about this permanent action below.

https://serverfault.com/questions/1014754/cause-of-warning-powershell-detected-that-you-might-be-using-a-screen-reader-an

When using UiPathOrch, it is recommended to always import PSReadLine.

To invoke the history of executed commands

In the PowerShell console, you can press the 'Up' arrow key to recall the history of commands

you have executed so far. Additionally, while typing a command, input prediction may appear in a lighter color. To accept this suggested text, press the 'Right' arrow key on the cursor.

About Wildcards

Some of the parameters that you specify to PowerShell cmdlets accept wildcards. Some of the wildcards available in PowerShell are:

- * (asterisk)
  Matches any text of any length.
- ? (Question Mark)
  Matches any single character.
- [] (brackets)
  It matches any one of the characters enclosed in square brackets. Multiple characters can be specified inside the brackets. This is similar to the square brackets of a regular expression.

To interpret the above characters literally, without using them as wildcards, escape them with a ` (backtick) just before them. For example, to display the asset my*asset with an asterisk in its name:

*Get-OrchAsset my`*asset*

Note that when you enter a name with the completion function, wildcard characters are automatically escaped.

About whitespace characters

To specify text that contains space characters in the parameter value, enclose the text in single quotes. If you want to include a single quote in the text enclosed in a single quote, escape the single quote with a single quote. For example, to create a new asset, my asset, and specify that's it as its value:

Set-OrchAsset Text 'my asset' 'that''s it'

Parameters available on many cmdlets

Specify the folder you want to operate in

- -Path

Specify the directory (folder) to be operated on. The -Path parameter of the UiPathOrch cmdlet supports wildcards. You can also specify multiple folders, separated by commas.

- -Recurse

A switch parameter that specifies that in addition to the current folder, the operation should also be performed on all subfolders, including descendant folders.

- -Depth

Specify the depth of the subfolder you want to work with. For example, 1 covers the current folder and immediate subfolders.

Hint: When using the UiPathOrch cmdlet, it's important to specify options such as -path, -recurse, and -depth immediately after the cmdlet name, before any other parameters. This ensures that the completion of subsequent parameters works correctly.

- -Name

Specify the entity you want to work with. The -Name parameter of the UiPathOrch cmdlet supports wildcards. You can also specify multiple names separated by commas.

Hint: For most UiPathOrch cmdlets, the -Name parameter is a zero-positional parameter, so typing this parameter name is optional.

Risk Mitigation Parameters

- -Whatif

Switch parameter available for cmdlets that Add, Update, and Remove entities. Show what operations are performed without actually adding, updating, or removing. For example, if you specify a parameter value with a wildcard, you can see how the wildcard expands. This is a very important and useful parameter for using UiPathOrch cmdlets.

- -Confirm

This switch parameter displays a confirmation message just before interacting with each entity and asks the user if they really want to perform the operation. Cmdlets featuring the -WhatIf parameter, as described above, also include the -Confirm parameter.

Common Parameters

Common parameters are available for all cmdlets, enhancing their usability and flexibility. Although there are several common parameters, this document focuses on introducing only two of them.

https://learn.microsoft.com/ja-jp/powershell/module/microsoft.powershell.core/about/about_commonparameters

- -Verbose

Normally, when you execute a cmdlet, what operations were attempted is not displayed in the console. To instruct it to display this information in the console (similar to the -WhatIf parameter), specify the -Verbose parameter. The alias for this parameter is -vb.

- -ErrorAction

This parameter specifies what to do with subsequent processing when multiple processing targets are specified (separated by wildcards and commas) and an error occurs during the process. The default value is Continue, which means that even if an error occurs, a message is displayed and subsequent processing is continued. If you specify Stop here, if an error occurs, a message will be displayed and it will be stopped. Subsequent processing is canceled. The alias for this parameter is -ea.

Limit the number of output rows

- -Skip

Show the specified number of lines, skipping from the beginning. For example, -Skip 5 skips the first five lines. This is useful when the output has a large number of lines.

- -First

Show only the specified number of rows. It can be used in combination with -Skip described above.

Processing the output of a cmdlet

Each cmdlet outputs a single entity on a single line. Each entity contains multiple pieces of data as fields, but by default only the fields defined in the default view are output in the console. There are several ways to process the output result:

- *Get-OrchAsset | Format-List*

  By default, each entity that the cmdlet outputs is displayed in the console in a Table format (the *Format-Table* cmdlet). However, the Table format limits the fields that can be output, and does not display all the fields contained in the entity. If you output in List format, you can see the contents of many fields.

- *Get-OrchAsset | select Name,Value*

  To output only a specified field in Table format, use the *Select-Object* cmdlet. *select* is the alias for *Select-Object*. The field name specified in Select can be completed.

- *Get-OrchAsset | ConvertTo-Json*

  Some of the entity fields that the cmdlet outputs are nested. For example, an asset entity contains a nested field with asset values for each user machine. These values cannot be

checked even in List-style views or selects. You can convert these entities to Json format with the *Convert-Json* cmdlet to see all nested field values.

- ***Get-OrchAsset -ExpandUserValue***

  Some cmdlets provide switch parameters to expand the contents of nested fields and print them to the console. *Get-OrchAsset*'s -ExpandUserValue expands and displays all values per user machine. For similar purposes, *Get-OrchRole* provides the -ExpandPermission parameter, and *Get-OrchAuditLog* provides the -ExpandEntity parameter.

- ***Get-OrchAsset / ? <columnName> -eq <value>***

  To filter the output only for entities that match the condition, pipe to *?*. For example, to get all assets that have the value False, use *Get-OrchAsset / ? {$_. Value -eq $false}. ?* is the alias for *Where-Object*. -eq means equal. Note that many UiPathOrch cmdlets have parameters to filter entities on the server side. In general, server-side processing is less taxable on the network. On the client side, however, you can filter by more flexible conditions. You can also run a combination of both.
  https://learn.microsoft.com/ja-jp/powershell/scripting/samples/removing-objects-from-the-pipeline--where-object-
  https://learn.microsoft.com/ja-jp/powershell/module/microsoft.powershell.core/about/about_simplified_syntax

- ***Get-OrchAsset > c:output.txt***

  > indicates the redirection output to a file. The above creates a output.txt in the current location of the C: drive. To move to the current location of the C: drive, cd c:. To create a output.txt in the root folder of the C: drive, > c:¥output.txt.

- ***Get-OrchAsset -ExportCsv c:***
- ***Get-OrchAsset -ExportCredentialCsv c:***

  The *Get-OrchAsset* cmdlet has parameters that instructs to export assets and credential assets to a CSV file. These CSV files can be imported with *Set-OrchAsset* and *Set-OrchCredentialAsset*. This method will be described later.

- ***Get-OrchLog <JobId> | Export-Csv C:¥<output-path>¥log.csv -Encoding utf8BOM***

  For cmdlets that do not support the -ExportCsv parameter, you can output a CSV file by piping the output to the *Export-Csv* cmdlet.

- ***Get-OrchJob / select ReleaseName,JobPriority,StartTime / Export-Csv C:¥<output-path>¥log.csv -Encoding utf8BOM***

  Export-Csv can be used in conjunction with the select described above. You can flexibly configure the items in the CSV file to be output.

About Providers

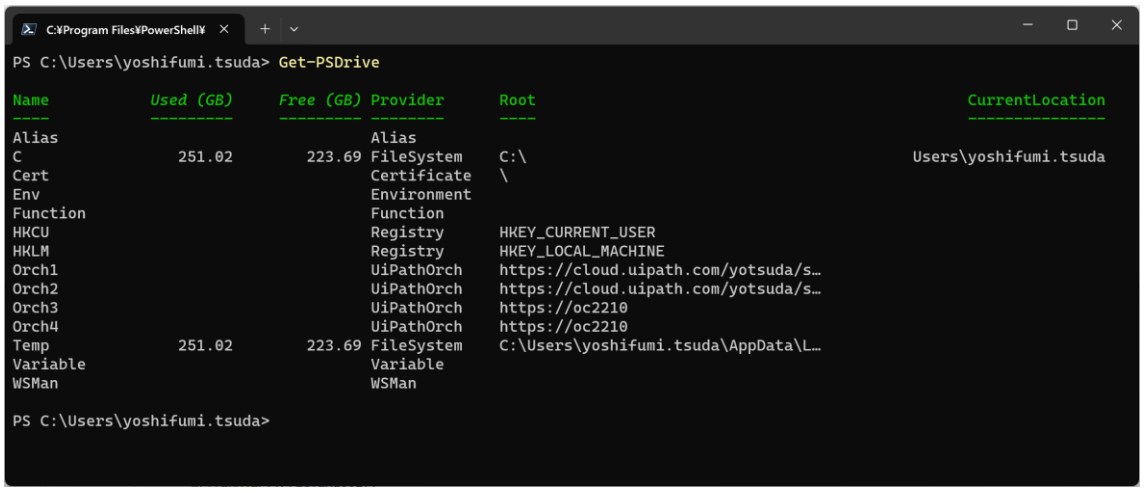
The PowerShell provider mounts the data in the tree structure as a PS drive and allows you to navigate this data with commands such as *cd* and *mkdir*. Examples of standard providers include:

- FileSystem
  Manipulate directories and files contained on external storage devices such as hard disks.
- Registry
  Manipulate the Windows registry database. The registry also has a tree structure, so you can operate it comfortably with PowerShell with unified commands.
- Variable
  Manipulate environment variables in Windows. Environment variables do not have a tree structure, but you can manipulate them with a unified set of commands on the PowerShell console.

Hint: UiPathOrch is also a PowerShell provider. You can work with modern folders in Orchestrator in the same way as you would with a file system or registry.

Confirm Mounted PSDrive

A PSDrive (PowerShell Drive) is a provider-mounted drive. You can use the *Get-PSDrive* cmdlet to see a list of PSDrives that are currently mounted.

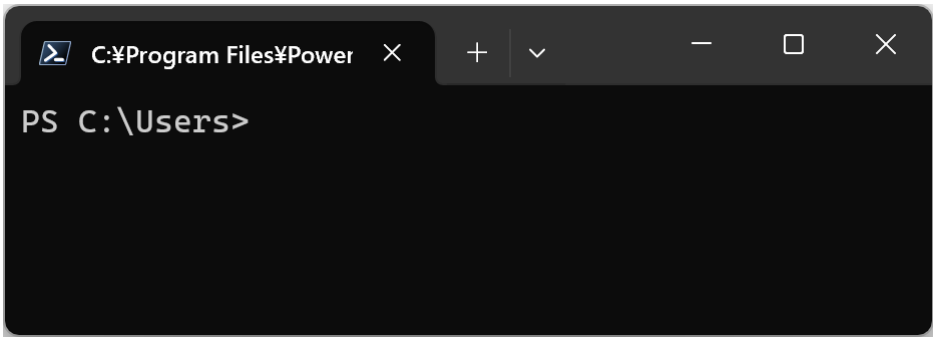


Hint: Normally, PSDrive is mounted with the *New-PSDrive* cmdlet, but to mount a PSDrive on UiPathOrch, you need to write a configuration file. This step is described later.

Hint: To clear the console, run *cls*. This stands for Clear Screen.

Working with Locations

PowerShell cmdlets process the current location (current location) by default. The current location of the current drive is displayed at the PowerShell console prompt.
For example, the current location of the following PS consoles is C:¥Users:



The current location is maintained on a per-drive basis. If you want the cmdlet to operate on a folder other than the current location, specify the location with the -Path parameter. Note that if you specify only a drive name for the -Path parameter, you are specifying the current location of that drive.
https://learn.microsoft.com/ja-jp/powershell/scripting/samples/managing-current-location

Path examples	Description
.	Current folder of the current drive
..	Parent folder of the current folder of the current drive
Orch1:	Current folder of the Orch1: drive
Orch1:¥	Root folder of the Orch1: drive
Orch2:	Current folder of the Orch2: drive
Orch2:¥	Root folder of the Orch2: drive
Orch2:..	Parent folder of the current folder of the Orch2: drive
Orch2:..¥..	Parent's parent folder of the current folder of the Orch2: drive
Orch1:sub	Subfolder under the current folder of the Orch1: drive
Orch1:¥sub	Subfolder under the root folder of the Orch1: drive

Example

- *Get-PSDrive*

Confirm the current location of each PS drive.

- *Get-Location*

Confirm the current location of the current drive. This alias is *pwd*. It stands for Print Working Directory.

- *Set-Location c:*

Specifying a drive name to the *Set-Location* cmdlet takes you to the current location of each drive. The alias for this is *cd*. It stands for Change Directory.

- cd c:¥

Navigate to the root folder of the specified drive. The root folder is specified by the ¥ symbol.

- *cd ..*

To navigate to the parent folder of your current location, use '..' (double period). To refer to your current folder, use '.' (single period).

- *Push-Location <folder path you want to move to>*

Records your current location and then moves you to a different folder. If you subsequently run *Pop-Location*, you will return to the original (recorded) location. The aliases for these commands are *pushd* and *popd*, respectively.

- *Get-ChildItem*

Show items (subfolders and files) in the current location. This alias is *dir*. It stands for directory.

- *dir -Recurse*

Show the current location and all items in its subfolders.

- *dir -Depth 1*

Show the current location and the items in its immediate subfolders.

- *ii .*

Open the current location in File Explorer. This is the alias for *Invoke-Item*.

Hint: In general, there are containers (folders) and non-containers (files) that can be handled by a provider, and the *Get-ChildItem(dir)* command displays both. However, in the UiPathOrch provider, *Get-ChildItem* only displays containers (folders). To view non-container items (such as users, machines, assets, and triggers), use the cmdlet corresponding

to each entity. For example, cmdlets include *Get-OrchUser, Get-OrchMachine*, Get-OrchAsset, *Get-OrchTrigger* , and etc.

How to operate on all UiPathOrch drives in bulk

You may want to mount multiple UiPathOrch drives and operate on them in bulk. For instance, to retrieve processes from all folders of the mounted UiPathOrch drives, you would do as follows:

Get-OrchProcess -Recurse -Path Orch1:¥,Orch2:¥,Orch3:¥

However, when you have a large number of mounted UiPathOrch drives, manually enumerating the drive names as above can be cumbersome. This can be accomplished by enumerating with the `Get-PSDrive` cmdlet as follows:

```
# Retrieve a list of UiPathOrch provider drive names and store them in a variable
$drivePaths = (Get-PSDrive -PSProvider UiPathOrch) | ForEach-Object { "$($_.Name):¥" }

# Retrieve processes from each drive and save them to a CSV file
Get-OrchProcess -Recurse -Path $drivePaths | Export-Csv -Path c:¥tmp¥all-processes.csv
```

A sample script for retrieving processes from all mounted tenants using the above method is available in the UiPathOrch installation folder.
Examples¥Get-ProcessesFromAllOrchDrives.ps1

About the UiPathOrch cmdlets

- Essentially, Orchestrator tenants don't have a root folder. However, for UiPathOrch providers, the root folder is available. This is useful when working with tenant entities or when working with multiple folders using the -Recurse parameter or wildcards on -Path parameter.
- In the parameters of each cmdlet, such as -Path, -Recurse, and -Depth, specify the folder to operate on. If these are not specified, the operation target is the current location.
- When you use parameters such as -Path, -Recurse, and -Depth, specify them immediately after the cmdlet name. This ensures that subsequent parameter value completion works properly.
- The folder separator in Orchestrator is / (forward slash), but UiPathOrch displays the separator as ¥ (backslash). However, you can also use / as a delimiter when specifying a folder path with the -Path parameter to the cmdlet.

Hint: The folder separator for the C: drive (FileSystem provider) is also displayed as ¥, but you can use both ¥ or / when specifying it in the cmdlet.

- Completion works for cmdlet names, parameter names, and parameter values.
  - To complete the cmdlet name, type get-orch[Ctrl+Space]. In addition to Get, there are cmdlets that begin with verbs such as set, add, and remove.
  - To complete the parameter name, type the cmdlet name and a space, followed by -[Ctrl+Space].
  - To complete a parameter value, type the cmdlet name, a space, a parameter name, a space, and then type Ctrl+Space. If you enter a value for a positional parameter, you can omit the parameter name.
  - When completing parameter values, you can narrow down the list of suggestions displayed by entering text containing wildcards and then pressing [Ctrl+Space].

List of cmdlets

Entity types	Cmdlet name (alias)	Overview
Folder	*Get-Location (pwd)*	Show Current Folder
	Set-Location (cd)	Change Current Folder
	Get-ChildItem (dir)	Show Subfolders
	New-Item (mkdir)	Create Folders
	Move-Item (mv)	Move Folders
	Copy-Item (copy)	Copy Folders
	Rename-Item (ren)	Rename Folder
	Remove-Item (rm)	Remove Folders
	Invoke-Item (ii)	Opening a Folder in a browser
	Get-OrchFolderUsage	Show Folders' usage information
Library	*Get-OrchLibrary*	Get the latest library packages
	Get-OrchLibraryVersion	Get Library Package Versions
	Copy-OrchLibrary	Copy Library Packages
	Remove-OrchLibrary	Remove Library Packages
	Import-OrchLibrary	Upload Library Packages
	Export-OrchLibrary	Download Library Packages
Package	*Get-OrchPackage*	Get the latest process packages
	Get-OrchPackageVersion	Get Process Package Versions
	Copy-OrchPackage	Copy Process Packages

	Remove-OrchPackage	Remove Process Packages
	Import-OrchPackage	Upload Process Packages
	Export-OrchPackage	Download Process Packages
Personal Workspace	*Get-OrchPersonalWorkspace*	Get Personal Workspaces
	Remove-OrchPersonalWorkspace	Remove Personal Workspaces
License	*Get-OrchLicenseNamedUser*	Get Named User Licenses
	Get-OrchLicenseRuntime	Get Runtime Licenses
	Enable-OrchLicenseRuntime	Enable Runtime Licenses
	Disable-OrchLicenseRuntime	Disable Runtime Licenses
Statistics	*Get-OrchJobStats*	Get Job Statistics
	Get-OrchLicenseStats	Get License Statistics
Job	*Open-OrchJob*	Open Job details in the browser
	Get-OrchJob	Get Jobs
	Start-OrchJob	Start Jobs
	Stop-OrchJob	Terminate Jobs
	Open-OrchJob	Open Jobs in the browser
Job Recordings	*Get-OrchJobVideo*	Get Jobs that have video attached
	Get-OrchJobMedia	Get Jobs that have screenshots attached
	Export-OrchJobMedia	Save Job Recordings screenshots to a file
	Remove-OrchJobMedia	Remove recordings screenshots from Jobs
Log	*Get-OrchLog*	Get Job Execution Logs
	Get-OrchAuditLog	Get Audit Logs
Process	*Get-OrchProcess*	Get Processes Assigned to Folders
	Add-OrchProcess	Add Processes to Folders
	Update-OrchProcess	Update Processes in Folders
	Remove-OrchProcess	Remove Process from Folder
	Edit-OrchProcess	Edit Process in the browser
	Update-OrchProcessVersion	Update the version of a process assigned to a folder
	Reset-OrchProcessVersion	Rollback the version of a process assigned to a folder
Robot	*Get-OrchRobot*	Get Robots
	Add-OrchRobotAccount	Add Robot Accounts
Calendar	*Get-OrchCalendar*	Get Calendars
	Copy-OrchCalendar	Copy Calendars to other tenants

	Remove-OrchCalendar	Remove Calendars
User	*Get-OrchUser*	Get Users/Groups
	Add-OrchUser	Add Users/Groups
	Copy-OrchUser	Copy Users between tenants
	Remove-OrchUser	Remove Users/Groups
	Add-OrchRoleToUser	Add Roles to Users
	Remove-OrchRoleFromUser	Remove Roles from Users
	Get-OrchCurrentUser	Get the current user connected to the tenant in this PowerShell session
	Update-OrchCurrentUserURPassword	Update the current user's Unattended Robot password
Role	*Get-OrchRole*	Get Roles
	Remove-OrchRole	Remove Roles
	Copy-OrchRole	Copy Roles between tenants
Folder User	*Get-OrchFolderUser*	Get Users Assigned to a Folder
	Add-OrchFolderUser	Assign Users to Folders
	Remove-OrchFolderUser	Remove Users from Folder
	Copy-OrchFolderUser	Copy (Assign) users who are assigned to a certain folder to other folders
	Add-OrchRoleToFolderUser	Add Roles to user assigned to folder
	Remove-OrchRoleFromFolderUser	Remove Roles from User assigned to folder
Machine	*Get-OrchMachine*	Get Machines
	Add-OrchMachine	Add Machines
	Copy-OrchMachine	Copy Machines between tenants
	Remove-OrchMachine	Remove Machines
Folder Machine	*Get-OrchFolderMachine*	Get Machines Assigned to a Folder
	Add-OrchFoderMachine	Assign Machines to Folder
	Copy-OrchFolderMachine	Copy (Assign) machines that are assigned to a certain folder to other folders
	Remove-OrchFolderMachine	Remove Machines from Folder
Asset	*Get-OrchAsset*	Get Assets Credential Assets
	Set-OrchAsset	Adding, updating, and removing Assets
	Set-OrchCredentialAsset	Adding, updating, and removing Credential Assets
	Copy-OrchAsset	Copy Assets/Credential Assets to other folders

	Remove-OrchAsset	Remove Assets
Credential Store	*Get-OrchCredentialStore*	Get Credential Stores
	Remove-OrchCredentialStore	Remove Credential Stores
Webhook	*Get-OrchWebhook*	Get Webhooks
	Copy-OrchWebhook	Copy Webhooks
	Remove-OrchWebhook	Remove Webhooks
	Enable-OrchWebhook	Enable Webhooks
	Disable-OrchWebhook	Disable Webhooks
Queue	*Get-OrchQueue*	Get Queues
	Add-OrchQueue	Add Queues
	Update-OrchQueue	Update Queues
	Copy-OrchQueue	Copy Queues
	Remove-OrchQueue	Remove Queues
	Get-OrchQueueItem	Get Queue Items
	Import-OrchQueueItem	Upload CSV files and bulk add Queue Items
Trigger	*Get-OrchTrigger*	Get Triggers
	Remove-OrchTrigger	Remove Triggers
	Copy-OrchTrigger	Copy Triggers
	Enable-OrchTrigger	Enable Triggers
	Disable-OrchTrigger	Disable Triggers
API Trigger	*Get-OrchApiTrigger*	Get API Triggers
	Remove-OrchApiTrigger	Remove API Triggers
	Copy-OrchApiTrigger	Copy API Triggers
	Enable-OrchApiTrigger	Enable API Triggers
	Disable-OrchApiTrigger	Disable API Triggers
Storage Bucket	*Get-OrchBucket*	Get Storage Buckets
	Get-OrchBucketItem	Get Storage Bucket Items
	Remove-OrchBucket	Remove Storage Buckets
	Copy-OrchBucket	Copy Storage Buckets
Test Case	*Get-OrchTestCase*	Remove Test Set
	Remove-OrchTestCase	Remove Test Cases
Test Set	*Get-OrchTestSet*	Get Test Sets
	Start-OrchTestSet	Start Test Sets
	Remove-OrchTestSet	Get Test Cases
Test Set Execution	*Get-OrchTestSetExecution*	Get Test Set Executions
	Stop-OrchTestSetExecution	Stop Test Set Executions

Test Schedule	*Get-OrchTestSetSchedule*	Get Test Schedules
	Copy-OrchTestSetSchedule	Copy Test Schedules
	Remove-OrchTestSetSchedule	Remove Test Schedules
Test Data Queue	*Get-OrchTestDataQueue*	Get Test Data Queues
	Copy-OrchTestDataQueue	Copy Test Data Queues
	Remove-OrchTestDataQueue	Remove Test Data Queues
Test Data Queue Item	*Get-OrchTestDataQueueItem*	Get Test Data Queue Items
Alert	*Get-OrchAlert*	Get Alerts
Session	*Get-OrchUserSession*	Get User Sessions
	Get-OrchMachineSession	Get Machine Sessions
Calendar	*Get-OrchCalendar*	Get Calendars
	Remove-OrchCalendar	Remove Calendars
	Copy-OrchCalendar	Copy Calendars
Platform Management	*Get-OrchPmUser*	Get Users
	Remove-OrchPmUser	Remove Users
	Get-OrchPmRobotAccount	Get Robot Accounts
	Set-OrchPmRobotAccount	Create/Update Robot Accounts
	Copy-OrchPmRobotAccount	Copy Robot Accounts to other tenants
	Remove-OrchPmRobotAccount	Remove Robot Accounts
	Get-OrchPmGroup	Get Groups
	Add-OrchPmGroup	Add Groups
	Remove-OrchPmGroup	Remove Groups
	Add-OrchPmMemberToPmGroup	Add members to Groups
	Remove-OrchPmMemberToFromPmGroup	Remove members from Groups
	Get-OrchPmExternalApiResource	Get External API Resources
	Get-OrchPmExternalApplication	Get External Applications
Document Understan	*Get-DuProject*	Get Projects
	Get-DuDocumentType	Get Document Types

ding		
Misc	*Clear-OrchCache*	Clear UiPathOrch in-memory cache
	Edit-OrchConfig	Open the UiPathOrch configuration file
	Get-OrchPSDrive	List UiPathOrch drives
	Set-OrchLocation	Move to the UiPathOrch installation directory

Exporting to CSV File

As mentioned at [Processing the output of a cmdlet], the output of all Get-OrchXxx cmdlets can be exported in any desired format using the following method. You can use [Ctrl+Space] to auto-complete column names. For the file path, you need to manually enter the drive name (e.g., c:), but after typing c:, pressing [Ctrl+Space] will auto-complete the folders.

```
PS> Get-OrchXxx | select <Column#1>,<Column#2> | Export-Csv <FilePath>
```

Additionally, the following cmdlets are equipped with the -ExportCsv parameter, which allows for easy export in an importable format. At this time, you can specify the CSV file encoding using the -CsvEncode parameter.

Cmdlets with the -ExportCSV Parameter	Cmdlets that can import the resulting CSV
dir -ExportCsv <file>	Import-Csv <file> | New-Item
Get-OrchMachine -ExportCsv <file>	Import-Csv <file> | Add-OrchMachine
Get-OrchProcess -ExportCsv <file>	Import-Csv <file> | Add-OrchProcess Import-Csv <file> | Update-OrchProcess
Get-OrchAsset -ExportCsv <file>	Import-Csv <file> | Set-OrchAsset
Get-OrchAsset -ExportCredentialCsv <file>	Import-Csv <file> | Set-OrchCredentialAsset
Get-OrchQueue -ExportCsv <file>	Import-Csv <file> | Add-OrchQueue Import-Csv <file> | Update-OrchQueue
Get-PmGroup -ExportCsv <file>	Import-Csv <file> | Set-PmGroup
Get-PmRobotAccount -ExportCsv <file>	Import-Csv <file> | Set- OrchPmRobotAccount

To output all entities of the tenant, please run each of the above Get cmdlets with the -Recurse parameter at the root folder.
Please note that dir and New-Item only work when the current folder is a UiPathOrch drive.

Importing a CSV file

Most UiPathOrch cmdlets allow you to specify parameters by importing from a CSV file. Generally, in PowerShell cmdlets, you can specify the values of each parameter by redirecting the output of the Import-Csv cmdlet via a pipeline. For example, you can do the following:

```
PS> Import-Csv <file name> | Copy-OrchAsset
```

In this case, the column values of the CSV file are passed to parameters with the same name as the columns. For instance, when you import the following .csv file into the Copy-OrchAsset cmdlet:

Path	Name	Destination
Orch1:¥Shared	Asset#1	Orch2:¥Shared
Orch1:¥Folder	Asset#2	Orch2:¥AnotherFolder

You get the same result as executing the following commands in sequence:

```
PS> Copy-OrchAsset -Path Orch1:¥Shared -Name Asset#1 -Destination Orch2:¥Shared
PS> Copy-OrchAsset -Path Orch1:¥Folder -Name Asset#2 -Destination Orch2:¥AnotherFolder
```

Additionally, the parameter values specified in the CSV file can be overridden by cmdlet parameters. For example, the following command imports all the assets listed in the CSV into the current folder:

```
PS Orch2:¥Shared> Import-Csv <file> | Set-OrchAsset -Path .
```

Copying Entities

UiPathOrch includes many cmdlets for copying entities. Cmdlets that copy tenant entities copy each entity to another tenant. Cmdlets that copy folder entities copy each entity to another folder within the same tenant or to a folder in another tenant.

Here, we'll explain how to use these cmdlets using `Copy-OrchCalendar` as an example. Other cmdlets can be used in a similar manner.

The following command copies the calendar with the specified name from the Orch1: drive

to the Orch2: drive.

```
PS Orch1:¥> Copy-OrchCalendar <calendar name> Orch2:
```

You can specify multiple names as a comma-separated list, including text with wildcards. The following command copies all calendars from the Orch1: drive to the Orch2: drive.

```
PS Orch1:¥> Copy-OrchCalendar * Orch2:
```

When the current drive is different, use the `-Path` parameter to specify the source. The following command copies all calendars from the Orch1: drive to the Orch2: drive.

```
PS C:¥> Copy-OrchCalendar -Path Orch1: * Orch2:
```

Specifying the Source Folder

The source folder is specified using the -Path parameter. If -Path is not specified, the current folder is used as the source. For cmdlets that copy folder entities, you can specify the source with the `-Path` as well as `-Recurse` and `-Depth` parameters. It is convenient to specify these parameters immediately after the cmdlet name. Specifying them before other parameters ensures that auto-completion for other parameters works correctly.

The `-Recurse` parameter copies entities in the subfolders of the source to the corresponding subfolders in the destination, assuming both have the same folder structure.

For example, the following command copies all assets from the Orch1: tenant to the Orch2: tenant, where both have the same folder structure:

```
PS Orch1:¥> Copy-OrchAsset -Recurse * Orch2:¥
```

Please note that some cmdlets may not yet support the `-Recurse` parameter. Additionally, when copying folder entities, information such as tags and links contained in each entity will also be copied.

Copying Folders

The copy command (Copy-Item cmdlet) copies the specified folders. When doing so, it also automatically copies the folder entities contained within each folder, such as folder packages, processes, assets, queues, etc.

The following command copies all folders from the Orch1: tenant to the Orch2: tenant:

```
PS Orch1:¥> copy -Recurse * Orch2:
```

Please note that when you copy classic folders, they are automatically converted to modern folders.

Copying Personal Workspaces

You can operate on personal workspaces, including those of other users, in UiPathOrch, provided that the exploration has been started. Since starting exploration is not supported via Orchestrator API, it must be done manually through the Orchestrator Web. After starting the exploration, execute the Clear-OrchCache cmdlet to reflect the start of the exploration in the PowerShell console.

Migrating a Tenant

Using the copy cmdlets described above, you can easily migrate a tenant. Additionally, it is easy to aggregate multiple tenants into a single tenant. There is no need to create CSV files; using wildcards allows you to copy all entities at once easily.

You can also copy entities from one folder to another. Furthermore, since each copy cmdlet supports importing from CSV files, you can plan which entities to copy to which folders in advance using a CSV file, and then complete the copy by importing that file. Many of the copy cmdlets have the parameters Path, Name, and Destination, so be sure to include these columns in your CSV file.

Tenant Migration Procedure

When migrating a tenant, it is important to copy each entity in the proper order. For example, copy packages before copying processes, as the process copy will fail if the package does not exist. You should copy all tenant entities before copying folders. It is recommended to execute each cmdlet in the following order.

```
PS Src:¥> Copy-OrchLibrary * * Dst:¥
PS Src:¥> Copy-OrchPackage * * Dst:¥
PS Src:¥> Copy-OrchCredentialStore * Dst:¥
```

PS Src:¥> Copy-OrchRole * Dst:¥
PS Src:¥> Copy-OrchUser * Dst:¥
PS Src:¥> Copy-OrchMachine * Dst:¥
PS Src:¥> Copy-OrchCalendar * Dst:¥
PS Src:¥> Copy-OrchWebhook * Dst:¥
(Manually start exploration in Orchestrator web for personal workspaces that need to be copied)
PS Src:¥> Clear-OrchCache
PS Src:¥> copy -Recurse * Dst:¥

Important Considerations for Tenant Migration

When migrating a tenant, be especially careful not to confuse the drive names of the source and destination. It is advisable to rename the drive names to something unmistakable, such as Src and Dst, before proceeding with the work. Additionally, ensure that the scopes for the Src drive are set to Read-only to prevent accidental deletion of entities from the Src drive.

Cmdlet Reference

In this document, the cmdlet names and parameters are listed as follows:

Get-ChildItem [[-Path] <string[]>] [[-Filter] <string>] [-Include <string[]>] [-Exclude <string[]>] [-Recurse] [-Depth <uint>] [-Force] [-Reload] Required permissions: OR. Folders.Read

- The brackets [] are optional. If the parameter name is enclosed in square brackets [], it means that it is a positional parameter and the parameter name can be omitted.
- The curly braces <> should be replaced with specific values. For example, <string> part must specify a specific string (text) value.
- <string[]> is an array of string (text) values. You can specify multiple values here, separated by commas. <uint> is an unsigned integer.
- The one that specifies only the parameter name (that is, the one that does not have a parameter value) is a switch parameter. In the example above, -Recurse, -Force are the switch parameters.
- The required permissions are those that must be specified in the configuration file to run the cmdlet. For example, OR.Folders.Read is permission to read folders in Orchestrator, and OR.Folders.Write is the permission to write folders. OR.Folders is a high-level

permission that encompasses both read and write permissions. Note that at least OR.Folders.Read is required to run any cmdlets.

Folders

Orchestrator folders can be manipulated using standard cmdlets. Parameters that are less frequently used or common are omitted for brevity. For a comprehensive list of exact parameters, refer to the Microsoft documentation.

- *Set-Location [[-Path] <string>]*
  Required permissions: OR. Folders.Read
  Change the current location. The alias is *cd*.

- *Get-Location*
  Required permissions: OR. Folders.Read
  Get the current location. The alias is *pwd*.

- *Get-Item [-Path] <string[]> [-Filter <string>] [-Include <string[]>] [-Exclude <string[]>] [-Force] [-Credential <pscredential>]*
- *Get-Item -LiteralPath <string[]> [-Filter <string>] [-Include <string[]>] [-Exclude <string[]>] [-Force] [-Credential <pscredential>]*
  Required permissions: OR. Folders.Read
  Get the folder details.

- *Get-ChildItem [[-Path] <string[]>] [[-Filter] <string>] [-Include <string[]>] [-Exclude <string[]>] [-Recurse] [-Depth <uint>] [-Force] [-Name] [- Reload]*
  Required permissions: OR. Folders.Read
  Show the subfolders of the folder specified by -Path. If -Path is not specified, the subfolders of the current folder are displayed. The alias is *dir*.

- *Invoke-Item [-Path] <string[]>*
  Required permissions: OR. Folders.Read
  Opens the specified location in a browser. The alias is ii.

- *New-Item [-Path] <string[]> [-WhatIf] [-Confirm] [-Description <string>] [-FeedType <OrchAPISession+FolderFeedType>]*
  Required permissions: OR. Folders.Write

Create folders. The alias is ni. Note that there is also a function *mkdir* that calls *New-Item*, but *mkdir* is not an alias, so the -Description parameter is not available on *mkdir*.

- ***Remove-Item [-Path] <string[]> [-Filter <string>] [-Include <string[]>] [-Exclude <string[]>] [-Recurse] [-Force] [-WhatIf] [-Confirm]***
  Required permissions: OR. Folders.Write
  Remove folders. The alias is *rmdir*.

- ***Move-Item [-Path] <string[]> [[-Destination] <string>] [-Filter <string>] [-Include <string[]>] [-Exclude <string[]>] [-PassThru] [-WhatIf] [-Confirm]***
  Required permissions: OR. Folders.Write
  Move folders. The alias is *mv*.

Change your current location

- ***cd orch2:***
  If you specify only a drive name, go to the current location of that drive.
- ***cd ¥***
  Go to the root folder of the current drive. '¥' stands for the root folder. It can also be cd / .
- ***cd orch2:¥***
  Go to the root folder of Orch2:.
- ***cd ..***
  Navigates to the parent folder of the current folder. '..' stands for the parent folder.
- ***cd ../..***
  Navigates to the parent folder of the parent of the current folder.
- ***cd [Ctrl+Space]***
  In completion, list the subfolders of the current folder and select them to move them.
- ***cd /[Ctrl+Space]***
  If you type / and then complete, folders directly under the root folder (regardless of the current folder) will be suggested as destinations.
- ***cd h****
  Go to a subfolder whose name starts with h. * is a wildcard that matches any text. If there are multiple matching folders, an error will occur.

View Folders

- *dir*
  Show the direct subfolders of the current folder. dir is another name for Get-ChildItem.
- *dir -Recurse*
  Show all descendant subfolders of the current folder. When run in the root folder, it displays all the folders in the tenant.
- *dir -Depth 2*
  Show all the children and grandchildren folders of the current folder. The Depth of the current folder is zero. Orchestrator's modern folder hierarchy is up to 7, so -Depth 6 behaves the same as -Recurse.
- *dir a**
  Show subfolders whose names begin with a.
- *dir -Path Orch1:*
  Orch1: Show the subfolder of the current location.
- *dir -Path Orch1:¥*
  Orch1: Show the subfolders of the root folder.
- *dir | ? { $_.FeedType -eq 'FolderHierarchy' }*
  Show only folders with feeds.
- *dir | ? { $_.ProvisionType -eq 'Manual' }*
  Show only classic folders.
- *gi . | Format-List*
  Display current folder info. gi is the alias for *Get-Item*.

Open Folders in the browser

- *ii .*
  Open the current folder in your browser. is the current folder (current location).
- *ii .,..*
  You can specify multiple folders at the same time, separated by commas. In the example above, the current folder and its parent folder are specified.

Make Folders

Folders can be created with *mkdir*. However, the *mkdir* command doesn't accept parameters such as -Description or -FeedType. To specify these, *New-Item* should be used.

- *mkdir hoge*
  Under the current folder, create a folder named hoge.
- *mkdir f1,f2,f3*
  Under the current folder, create three folders named F1, F2, and F3.
- *New-Item fuga -Description 'This is a new folder.'*
  Create a folder named fuga under the current folder and write This is a new folder. Set the text.
- *New-Item ¥yyy -FeedType FolderHierarchy*
  Create a folder with a feed named yyy under the root folder. Values that can be specified for -FeedType can be entered in completion. Note that folders with feeds can only be created directly under the root folder.

Copy Folders

Folders can be copied with *copy* command. c*opy* is an alias for *Copy-Item*. Folders can be copied within the same tenant or between different tenants. *copy* copies assigned users with roles and machines, along with the folder, during the copy process. When copying folders within the same tenant, the internal IDs of users, roles, and machines are duplicated as they are. However, when copying between different tenants, the command searches for users, roles, and machines in the target tenant by name and assigns their IDs to the copied folder. If an entity with the same name does not exist in the target tenant, the command displays a warning but continues the process.
Personal workspace folders will not be copied. Copying a classic folder will result in the destination folder becoming a modern folder. Folders with feeds located directly under the root folder can only be copied directly under the root folder of the same drive or a different drive. Non-top-level folders with feeds can be copied under any folder.

- *copy Orch1:¥* Orch2:¥ -Recurse*
  This command copies all folders and their subfolders from Orch1:¥ to Orch2:¥.
- *copy srcFolder dstFolder*
  This command copies the srcFolder located directly under the current folder to dstFolder located directly under the current folder, creating the dstFolder¥srcFolder folder.

Move Folders

You can only move folders between the same tenants. Please note moving folders between different tenants is not supported.

- *mv hoge fuga*

Move the hoge folder from the current folder to the fuga folder.

- *mv f1,f2 fuga*
  Move the f1 and f2 folders from the current folder to the fuga folder.
- *mv * fuga*
  Move all folders from the current folder to the fuga folder.
- *mv h* fuga*
  Move all folders that start with h from the current folder to the fuga folder.
- *mv h* ..*
  Move all folders from the current folder that start with h to the parent folder.
- *mv h* fuga -WhatIf*
  See what happens when you *run mv h* fuga*. The folders are not actually moved.

Rename a Folder

- *ren hoge hoehoe*
  Rename the hoge folder under the current folder to hoehoe.

Remove Folders

- *rmdir hoge*
  Remove the hoge folder under the current folder.
- *rmdir**
  Remove all folders under the current folder. Don't display a confirmation message and remove them immediately. This action cannot be undone, so proceed carefully.
- *rmdir h* -WhatIf*
  Remove all folders under the current directory that start with h. Using the -WhatIf parameter will not actually remove these folders, but will instead confirm the names of the folders that would be removed.
- *rmdir h* -Confirm*
  Remove all folders under the current folder that start with h. A confirmation message is displayed each time a folder is removed.

View Folders' usage information

Get-OrchFolderUsage cmdlet is implemented by calling a non-public OC Web API (not documented in the Swagger documentation), so it may not work with some versions of the Orchestrator.

- *Get-OrchFolderUsage*
  Displays usage information for the current folder.
- *Get-OrchFolderUsage -Recurse | Export-Csv c:¥temp¥usage.csv -Encoding sjis*
  Saves the usage information for the current folder and its subfolders to the file c:¥temp¥usage.csv.
- *Get-OrchPersonalWorkspace | Get-OrchFolderUsage*
  Displays usage information for all personal workspace folders. Personal workspace folders of others are not shown with the *dir* command, but by piping the output of *Get-OrchPersonalWorkspace* to *Get-OrchFolderUsage*, you can retrieve their usage information.

Library Packages

- *Get-OrchLibrary [[-Id] <string[]>] [-Path <string[]>]*
  Required permissions: OR.Execution.Read
  Get the latest version of the library in the tenant feed of the current Drive (tenant). The first parameter allows you to specify multiple library names, separated by commas and wildcards. If the name of the library is omitted, all libraries are enumerated. For the -Path parameter, specify the drive name.

- *Get-OrchLibraryVersion [[-Id] <string[]>] [-Path <string[]>]*
  Required permissions: OR.Execution.Read
  Retrieves all versions of libraries in the tenant feed of the current Drive (tenant). The first parameter allows you to specify multiple library names, separated by commas and wildcards. If the name of the library is omitted, all libraries are enumerated. For the -Path parameter, specify the drive names.

- *Remove-OrchLibrary [-Id] <string[]> [[-Version] <string[]>] [-Path <string[]>] [-WhatIf] [-Confirm]*
  Required permissions: OR.Execution.Write
  Remove a library in the tenant feed of the current Drive (tenant). The first parameter allows you to specify multiple library names, separated by commas and wildcards. In the next parameter, specify the version you want to remove. The name and version of the library cannot be omitted. Specifying * for the version removes all versions. Completion allows you to view the existing libraries and versions.

- *Import-OrchLibrary [-Source] <string[]> [-Path <string>] [-WhatIf] [-Confirm]*
  Required permissions: OR.Execution.Write
  Uploads the specified package files to tenant library feed. The -Source parameter accepts a comma-separated list of nupkg file names or directory names containing nupkg files on a local drive. Wildcards can be used.

- *Export-OrchLibrary [[-Id] <string[]>] [[-Version] <string[]>] [[-Destination] <string>] [-Path <string[]>] [-WhatIf] [-Confirm]*
  Required permissions: OR.Execution.Read
  Downloads the specified version of the library packages.

View the latest version of Libraries

- *Get-OrchLibrary*
  Show all packages in the library feed of the current drive (tenant).
- *Get-OrchLibrary -Path Orch1:,Orch2:*
  Show all packages in the library feed for the Orch1: drive and the Orch2: drive. Auto-completion feature is available when typing the drive names.
- *Get-OrchLibrary e**
  Show all packages in the current Drive (tenant) library feed with names that begin with e. You can complete the package name. You can also specify multiple package names, separated by commas.

View all versions of Libraries

- *Get-OrchLibraryVersion*
  Show all versions of packages in the library feed of the current drive (tenant).
- *Get-OrchLibraryVersion -Path Orch1:,Orch2:*
  Show all versions of packages in the library feed on the Orch1: drive and Orch2: drive. Auto-completion feature is available when typing the drive names.
- *Get-OrchLibraryVersion e**
  Show all versions of packages with names that start with e in the library feed of the current drive (tenant). Auto-completion feature is available when typing the package names. You can also specify multiple package names, separated by commas.

Remove Libraries

Note that even if a library package is used, it can be removed by executing the following.

- *Remove-OrchLibrary <library names> <versions>*
  Remove the specified version of the package with the specified library name in the library feed of the current drive (tenant). The package name and version number are not optional. Please fill in with the completion function. You can use wildcards in the package name and version number.
- *Remove-OrchLibrary *mylib* * -WhatIf*
  Remove all versions of libraries with mylib in their names in the library feed of the current drive (tenant). It doesn't actually remove it, it displays the libraries and versions to be removed.
- *Remove-OrchLibrary *mylib* * -Confirm*
  Remove all versions of libraries with mylib in their names in the library feed of the current drive (tenant). Each time you remove one, you will receive a confirmation message.

Upload Libraries

- *Import-OrchLibrary c:¥pkg¥*.nupkg*
  Uploads specified package files to the tenant library feed of the current drive.
- *Import-OrchLibrary c:¥pkg*
  If a folder is specified for the -Source parameter, all *.nupkg files contained in this folder will be uploaded. Note that the -Source parameter can simultaneously specify multiple folder names and file names, separated by commas.
- *Import-OrchLibrary c:¥pkg¥*.nupkg -Path Orch1:,Orch2:*
  Uploads specified package files to the tenant library feed of Orch1: and Orch2:.

Download Libraries

- *Export-OrchLibrary*
  Downloads all libraries from the current drive (tenant) and saves them to the current location on the C: drive. To open the current location on the C: drive in File Explorer, execute *ii c:*.
- *Export-OrchLibrary <library names>*
  Downloads all versions of the specified library and saves them to the current location on the C: drive.

- *Export-OrchLibrary <library names> <versions>*
  Downloads the specified version of the specified library and saves it to the current location on the C: drive.
- *Export-OrchLibrary <library names> -Destination c:¥lib*
  Downloads all versions of the specified library and saves them to C:¥lib.

Process Packages

Process packages are located in the tenant's process feed, as well as in folders with feeds. If you run Get-OrchPackage in a folder with a feed (and its subfolders), you will see the packages in that folder feed. If you run Get-OrchPackage in a folder that does not have a feed or in the root folder, it displays the packages in the tenant's process feed.

- *Get-OrchPackage [[-Id] <string[]>] [-Path <string[]>] [-Recurse]*
  Required permissions: OR.Execution.Read
  Get the latest version of the process package.

- *Get-OrchPackageVersion [[-Id] <string[]>] [-Path <string[]>] [-Recurse]*
  Required permissions: OR.Execution.Read
  Retrieves all versions of a process package.

- *Remove-OrchPackage [-Id] <string[]> [-Version] <string[]> [-Path <string[]>] [-Recurse] [-WhatIf] [-Confirm]*
  Required permissions: OR.Execution.Write
  Remove a specified version of a process package.

- *Export-OrchPackage [[-Id] <string[]>] [[-Version] <string[]>] [[-Destination] <string>] [-Path <string[]>] [-Recurse] [-WhatIf] [-Confirm]*
  Required permissions: OR. Execution.Read
  Downloads the specified version of the process packages.

View Available Packages in the Current Folder

- *Get-OrchPackage*
  If you run Get-OrchPackage in the root folder or in a folder that does not have a folder feed , you will see the packages in that tenant feed.

If you run Get-OrchPackage on a folder with a folder feed , the packages in that folder feed are displayed.

- *Get-OrchPackage <package names>*
  You can specify multiple package names, separated by wildcards and commas. It can also be specified as a completion input. Show only the specified packages.

View packages in the tenant feed for the current tenant

- *Get-OrchPackage -Path ¥*
  To view packages in the tenant feed, regardless of whether there is a feed in the current folder, specify the root folder for the -Path parameter.

View packages from all feeds in the current Drive

- *Get-OrchPackage -Recurse*
  In the root folder, run Get-OrchPackage with the -Recurse parameter to view this tenant feed and all packages in folders with feeds.
- *Get-OrchPackage -Path **
  Running Get-OrchPackage with -Path * in the root folder also gives the same result as above. This is because you can only create a folder with a feed directly under the root folder.
- *Get-OrchPackage -Recurse a**
  If you specify a package name with -Recurse, you can search for a folder feed that contains a process package with this name. If you fill in after -Recurse, the suggestions will show the package names in all feeds.

View All Versions of Packages

- *Get-OrchPackageVersion*
  Show all the versions of a package in the feed available in the current folder.
- *Get-OrchPackageVersion -Recurse e**
  Show all versions of packages with names that begin with e in the package and folder feeds of the current drive (tenant). Auto-completion feature is available when typing the package names. You can also specify multiple package names, separated by commas.
- *Get-OrchPackageVersion -Path Orch1:¥,Orch2:¥ -Recurse*
  Show all versions of packages in all feeds on the Orch1: drive and Orch2: drive. Auto-completion feature is available when typing the drive names.

Remove Packages

- *Remove-OrchPackage <package names> <versions>*
  Removes a specified package version from the feed available in the current folder. You can specify multiple package names and versions, including wildcards, separated by commas. It can also be complemented. Note that the active package version does not appear in the list of completion candidates.
- *Remove-OrchPackage -Recurse * * -WhatIf*
  Specifying * for both the package name and version number removes all inactive packages at once. If you specify the -Recurse parameter in the root folder, the tenant feed and all folder feeds will be processed together. Check the package to be removed with -WhatIf, and then execute it without specifying -WhatIf.

Upload Packages

- *Import-OrchPackage c:¥pkg¥*.nupkg*
  Uploads the specified package file to the process feed of the current folder. If the current folder does not have a process feed, it uploads to the tenant's package feed.
- *Import-OrchPackage c:¥pkg*
  When a folder is specified for the -Source parameter, it uploads all *.nupkg files contained in this folder. Note that the -Source parameter can simultaneously specify multiple folder names and file names, separated by commas.
- *Import-OrchPackage c:¥pkg¥*.nupkg -Path Orch1:¥,Orch2:¥myfolder*
  Uploads the specified package files to both the tenant process feed of Orch1: and the process feed of myfolder in Orch2:. If the Orch2:¥myfolder folder does not have a process feed, it uploads to the tenant process feed of Orch2:.
- *Import-OrchPackage c:¥pkg -Recurse*
  Running with the -Recurse parameter in the root folder will upload in bulk to the tenant feed and all folders with a feed. Note that only folders directly under the root folder can have a feed, so specifying the -Recurse option only makes sense when executed in the root folder.
- *Import-OrchPackage c:¥pkg -SourceRecurse*
  Specifying the -SourceRecurse switch parameter uploads *.nupkg files located in subfolders of the folder specified in -Source (in this case, c:¥pkg) to the feed of a modern folder with the same name. This parameter assumes that folders with the same name as personal workspace folders or folders with a feed exist under c:¥pkg. It is useful for

migrating packages downloaded with *Export-OrchPackage -Recurse* to another tenant in bulk.

Download Packages

- *Export-OrchPackage*
  Downloads all process packages from the feed of the current folder and saves them to the current location on the C: drive. To open the current location on the C: drive in File Explorer, execute *ii c:*. Note that when downloading packages from a folder with a feed, a subfolder with the same name will be automatically created in the destination folder.
- *Export-OrchPackage -Recurse*
  When executed in the root folder, it downloads all process packages from the tenant feed and all process packages from every folder with a feed.
- *Export-OrchPackage <Process Name> <Version Number> -Destination c:¥pkg*
  Downloads the specified version of the specified process to c:¥pkg.
- *Export-OrchPackage * 1.0.1 -WhatIf*
  Downloads version 1.0.1 of all packages to the current location on the C: drive. Specifying the version as 1.0.[1-3] will download versions 1.0.1, 1.0.2, and 1.0.3 of all packages. -WhatIf specifies to confirm which packages would be downloaded without actually performing the download.

Processes

- *Get-OrchProcess [[-Name] <string[]>] [-Path <string[]>] [-Recurse] [-Depth <uint>]*
  Required permissions: OR. Execution.Read.
  Gets the processes that have been assigned to a folder and are now available for execution.

- *Remove-OrchProcess [[-Name] <string[]>] [-Path <string[]>] [-Recurse] [-Depth <uint>] [-WhatIf] [-Confirm]*
  Required permissions: OR. Execution.Write
  Removes the processes assigned to a folder from a folder.

- *Update-OrchProcessVersion [-Name] <string[]> [[-Version] <string>] [-Path <string[]>] [-Recurse] [-Depth <uint>] [-WhatIf] [-Confirm]*
  Required permissions: OR. Execution.Write
  Updates the version of the process assigned to a folder. If no version is specified, update

to the latest version.

- *Reset-OrchProcessVersion [-Name] <string[]> [-Path <string[]>] [-Recurse] [-Depth <uint>] [-WhatIf] [-Confirm]*
  Required permissions: OR. Execution.Write
  Rolls back the version of the process assigned to the folder.

View Processes Assigned to a Folder

- *Get-OrchProcess*
  Show the processes assigned to the current folder.
- *Get-OrchProcess -Recurse*
  Show the processes assigned to the current folder and all its subfolders. When run in the root folder, it displays all processes from folders included in that tenant.
- *Get-OrchProcess -Recurse *proc**
  Search all folders for processes that contain proc in the process name.

Removing Processes from a Folder

- *Remove-OrchProcess <process names>*
  Remove a specified process from the current folder. The process name can be completed. You can also specify multiple process names, separated by commas.
- *Remove-OrchProcess a**
  Removes from the current folder all processes whose name starts with A.
- *Remove-OrchProcess a* -WhatIf*
  Show the processes that are targeted for deletion, without actually removing them.
- *Remove-OrchProcess -Recurse <process names>*
  Remove the specified process from its current location and all its subfolders. You can use wildcards in the process names.

Edit Processes in the browser

- *Edit-OrchProcess <process names>*
  Edit the specified process with the browser.

Update the Version of Processes Assigned to a Folder

- *Update-OrchProcessVersion <process names>*
  Updates the specified process to the latest version in the current folder. The process name can contain multiple names that contain wildcards, separated by commas. Completion input is also available. Completion suggests only processes that are not up to date.
- *Update-OrchProcessVersion <process names> <version>*
  Updates the specified process to the specified version in the current folder.
- *Update-OrchProcessVersion * -WhatIf*
  In the current folder, update all processes to the latest version. The -WhatIf parameter indicates that you don't actually want to update, but to display the process that you want to update.
- *Update-OrchProcessVersion -Recurse **
  Updates all processes in the current folder and all its subfolders to the latest version. When run in the root folder, it updates all processes that have the latest version available in all folders.
- *Update-OrchProcessVersion -Recurse <process names> <version>*
  Updates the specified process to the specified version in the current folder and all its subfolders.
- *Reset-OrchProcessVersion <process names>*
  Rolls back the version of a given process in the current folder. The process name can contain multiple names that contain wildcards, separated by commas. Completion input is also available.
- *Reset-OrchProcessVersion -Depth 1 <process names>*
  Rolls back the version of a given process in the current folder and its immediate subfolders.

Personal Workspaces

In drives connected using the settings of non-confidential app, the *dir* command (*Get-ChildItem* cmdlet) automatically displays the connected user's personal workspace. Additionally, personal workspaces of others that are being explored through the Orchestrator web interface are also displayed by the *dir* command. Personal workspace folders displayed by the dir command can be operated on just like any other folders. For example, it is possible to upload and download packages, as well as get and update assets. Note that the Orchestrator

Web API does not support starting/stopping exploration of personal workspaces. If you need to operate on someone else's personal workspace, first start exploring manually through the Orchestrator web interface. Afterwards, executing *Clear-OrchCache* in the PowerShell console followed by the *dir* command will display the personal workspaces.

- *Get-OrchPersonalWorkspace [[-Name] <string[]>] [-Path <string[]>]*
  Required permissions: OR.Folders.Read.
  Get the personal workspace folders on the current drive.

View Personal Workspaces

- *Get-OrchPersonalWorkspace*
  Displays all personal workspaces for this tenant, including those that are not being explored. However, you cannot access the contents of those personal workspaces.
- *Get-OrchPersonalWorkspace *name**
  Displays personal workspaces where the Name matches *name*.

Licenses

- *Get-OrchLicenseNamedUser [-RobotType] <string[]> [-Path <string[]>]*
  Required permissions: OR.License.Read
  Get Named User Licenses. Specify NonProduction, Attended, Unattended, Development, and etc to -RobotType parameter. You can specify multiple robot types with wildcards, separated by commas.
- *Get-OrchLicenseRuntime [[-RobotType] <string[]>] [-Path <string[]>]*
  Required permissions: OR.License.Read
  Get Named User Licenses. Specify NonProduction, Attended, Unattended, Development, and etc to -RobotType parameter. You can specify multiple robot types with wildcards, separated by commas.
- *Enable-OrchLicenseRuntime [-RobotType] <string[]> [-Key] <string[]> [-Path <string[]>] [-WhatIf] [-Confirm]*
  Required permissions: OR.License.Write
  Enable Runtime Licenses.
- *Disable-OrchLicenseRuntime [-RobotType] <string[]> [-Key] <string[]> [-Path <string[]>] [-WhatIf] [-Confirm]*
  Required permissions: OR.License.Write
  Disable Runtime Licenses.

View Named User Licenses

- *Get-OrchLicenseNamedUser*
  Show all Named User Licenses on this tenant.
- *Get-OrchLicenseNamedUser NonProduction*
  Show Named User Licenses of NonProduction.
- *Get-OrchLicenseNamedUser NonProduction,Attended*
  Show Named User Licenses of NonProduction and Attended.
- *Get-OrchLicenseNamedUser -Path Orch1: * | Export-Csv c:lnu.csv*
  Export all Named User Licenses on this tenant to nl.csv under the current location of C:. To open the lnu.csv, execute *c:lnu[Tabl][Enter]* . To open the current location of C: with file explorer, execute *ii c:[Enter]*.
- *Get-OrchLicenseNamedUser -Path Orch1:,Orch2 **
  Show all Named User Licenses on the specified drive (tenant).

View Runtime Licenses

- *Get-OrchLicenseRuntime*
  Show all Runtime Licenses on this tenant.
- *Get-OrchLicenseRuntime NonProduction*
  Show Runtime Licenses of NonProduction.
- *Get-OrchLicenseRuntime NonProduction,Attended*
  Show Rutime Licenses of NonProduction and Attended.
- *Get-OrchLicenseRuntime -Path Orch1: * | Export-Csv c:lr.csv*
  Export all Runtime Licenses on this tenant to nl.csv under the current location of C:. To open the nl.csv, execute *c:lr[Tabl][Enter]* . To open the current location of C: with file explorer, execute *ii c:[Enter]*.
- *Get-OrchLicenseRuntime -Path Orch1:,Orch2 **
  Show all Named User Licenses on the specified drive (tenant).

Enable Runtime Licensee

- *Enable-OrchLicenseRuntime Unattended <key>*
  Enables the license for <key> where RobotType is Unattended. Multiple RobotType and key can be specified, separated by commas, with wildcards. Autocomplete input is also available.

- *Enable-OrchLicenseRuntime * * -WhatIf*
  Enables all licenses that are disabled for all RobotTypes. -WhatIf indicates to confirm the RobotType and key that would be enabled without actually enabling them.

Disable Runtime Licensee

- *Disable-OrchLicenseRuntime Unattended <key>*
  Disables the license for <key> where RobotType is Unattended. Multiple RobotType and key can be specified, separated by commas, with wildcards. Autocomplete input is also available.
- *Disable-OrchLicenseRuntime * * -WhatIf*
  Disables all licenses that are disabled for all RobotTypes. -WhatIf indicates to confirm the RobotType and key that would be disabled without actually enabling them.

Jobs

In the Examples directory under the UiPathOrch installation directory %UserProfile%¥Documents¥PowerShell¥Modules¥UiPathOrch, there is a sample script that searches every 15 minutes for jobs that have been in Pending state for more than 15 minutes and notifies you by email if any.

- *Get-OrchJob [-Last <string>] [-CreationTimeAfter <datetime>] [-CreationTimeBefore <datetime>] [-Priority <string>] [-Process <string[]>] [-SourceType <string[]>] [-State <string[]>] [-Skip <ulong>] [-First <ulong>] [-Path <string[]>] [-Recurse] [-Depth <uint>]*
- *Get-OrchJob [[-Id] <long[]>] [-Skip <ulong>] [-First <ulong>] [-Path <string[]>] [-Recurse] [-Depth <uint>]*
  Required permissions: OR. Jobs.Read
  Get the job. In the first set of parameters, you can specify the conditions for the jobs you want to retrieve with different parameters. In the second parameter set, the -Id parameter can directly specify the ID of the job to be retrieved. The completion list for the -Id parameter only displays the Ids of the jobs that have been retrieved and cached in UiPathOrch's memory. Therefore, after retrieving multiple jobs in the first parameter set, if there is a job that you want to see in detail, specify its Id in the -Id parameter. Even if an Id exists in the cache, Get-OrchJob checks and displays the latest status of that job.

- *Start-OrchJob [-Name] <string[]> [[-RuntimeType] <string>] [[-JobsCount] <int>]*

[-Path <string[]>] [-WhatIf] [-Confirm]

Required permissions: OR. Jobs.Write

Start Job. The completion of the -Name parameter allows you to specify the processes that are assigned to the folder. Note that complex conditions (such as parameters and entry points) cannot be specified.

- *Stop-OrchJob [-Id] <Object[]> [-Force] [-Path <string[]>]*

  Required permissions: OR. Jobs.Write

  Stop the job. For the -Id parameter, specify the ID of the job checked by Get-OrchJob. If the -Force switch parameter is attached, the specified job will be killed.

View Jobs

You can also specify all of the following parameters at the same time.

- *Get-OrchJob*

  Show the jobs in the current folder. *Get-OrchJob* caches the job ID for completion input the next time the cmdlet is executed, but this cache is not used to output the execution result of *Get-OrchJob*. *Get-OrchJob* always queries Orchestrator to see the most up-to-date information.

  Note that the date and time data (CreationTime, StartTime, etc.) output by *Get-OrchJob* is in UTC, but the default table view converts them to local time and displays them. You can find this view definition in the OrchProvider.Format.ps1xml file in the installation directory of the UiPathOrch module.
- *Get-OrchJob -Recurse*

  Show all jobs in the current folder and its subfolders.
- *Get-OrchJob -Path ..*

  Show the jobs in the parent folder.
- *Get-OrchJob -Recurse*

  Show all jobs in the current folder and its subfolders.
- *Get-OrchJob -Last Day*

  Show the jobs for the last one day in the current folder. The value of the -Last parameter can be Hour, Day, Week, Month, and so on. In addition,
- *Get-OrchJob -State Pending,Suspended*

  Show jobs that are in a pending or suspended state. To display only failed jobs, specify Faulted for the -State parameter. This parameter value can be completed input.
- *Get-OrchJob -SourceType QueueTrigger*

Show jobs started by queue triggers.

- *Get-OrchJob -Process <process names>*
  Show the jobs for a given process. You can use wildcards in the process name. * to display the logs of all processes assigned to that folder. You can also specify multiple process names, separated by commas. The completion feature of the -Process parameter lists the names of the processes that are assigned to this folder.
  In addition, you can filter the jobs displayed by parameters such as -CreationTimeAfter, -CreationTimeBefore, and -Priority.
- *Get-OrchJob | ? { $_. CreationTime.ToLocalTime() -ge '2024/01/15 14:00:00' }*
  The date and time data specified for -CreationTimeAfter and -CreationTimeBefore is interpreted as local time. But the CreationTime that Get-OrchJob outputs is UTC, so in the pipe? (another name for Where-Object) and filter by CreationTime, manually convert it to local time as described above and then compare. It should also be noted that the default table view that displays the output results of Get-OrchJob converts UTC to local time and then displays it.
- *Get-OrchJob | ? StopStrategy -eq Kill*
  Show only killed jobs.

Group jobs and count them

- *Get-OrchJob | group ReleaseName -NoElement*
  Group by process name. Item names can be completion. Group is another name for Group-Object.
- *Get-OrchJob | group HostMachineName -NoElement*
  Group by machine name.
- *Get-OrchJob | group HostMachineName,State -NoElement*
  Group by machine name and status.
- *Get-OrchJob | group LocalSystemAccount,State -NoElement*
  Group by the executed account name.
- *Get-OrchJob | group { $_. StartTime.ToLocalTime(). Date } -NoElement*
  Group by start date.
- *Get-OrchJob | group { $_. StartTime.ToLocalTime(). ToString('yyyy/MM/dd') }*
  Group by start month.
- *Get-OrchJob | group { $_. StartTime.ToLocalTime(). DayOfWeek }*
  Group by the start day of the week.
- *Get-OrchJob | group { $_. StartTime.ToLocalTime(). DayOfWeek },State*

Group by start day of the week and status.

- *Get-OrchJob | ? { $_. StartTime.ToLocalTime(). DayOfWeek -eq 'Sunday' }*
  Only print the jobs that were executed on Sunday.
- *Get-OrchJob | group { $_. StartTime.ToLocalTime(). Hour },State*
  Group by the time slot and status you started. You can easily analyze trends by state and when jobs were running.
- *Get-OrchJob | group ReleaseName -NoElement | sort Count -Descending*
  Sort the grouped contents from highest to lowest number. sort is another name for Sort-Object.
- *Get-OrchJob -State Faulted | group ReleaseName | sort Count -Descending*
  Group only failed jobs by process name, from highest to lowest number. You can easily analyze which processes are failing more frequently.

Find the execution time of each job

- *Get-OrchJob | sort { ($_. EndTime - $_. StartTime). TotalSeconds } -Descending*
  Sort jobs from longest to least longest.
- *Get-OrchJob | select Id, ReleaseName, @{Name='TotalSeconds'; Expression={ ($_. EndTime - $_. StartTime). TotalSeconds }} | sort TotalSeconds -Descending*
  Along with the details of the job, it also outputs the execution time.
- *Get-OrchJob | select *, @{Name='TotalSeconds'; Expression={ ($_. EndTime - $_. StartTime). TotalSeconds }} | sort TotalSeconds*
  Together with the calculated execution time, all columns are also printed. Wildcards can be used to specify column names, so if you specify *, all column names will be output.
- *Get-OrchJob | % { ($_. EndTime - $_. StartTime). TotalMinutes } | Measure-Object -Average -Sum -Maximum -Minimum*
  Print the average, longest, and shortest job execution time. % is another name for ForEach-Object.

Start Jobs

- *Start-OrchJob <process names>*
  Start a job in the current folder. The process name can be completed.
- *Start-OrchJob <process names> Unattended*
  Immediately after the process name, you can specify the type of license (the -RuntimeType parameter). The above example runs the specified process with an Unattended license.

- *Start-OrchJob <process names> Unattended 3*
  You can specify the number of jobs (-JobsCount parameter) to start running.
- *Start-OrchJob <process names> -JobsCount 3*
  If you do not want to specify the type of license, but only the number of jobs, specify the name of the -JobsCount parameter.

Stop Jobs

- *Stop-OrchJob <job ids>*
  Stops the job with the specified job ID. The job ID displayed by the completion input of the job ID (-Id parameter) is only the ID of the job that has been obtained (cached) by Get-OrchJob and can be stopped (running). Note that if you manually enter the job ID without using completion input, it will work fine even if the job ID is not cached.
- *Stop-OrchJob <jobid>,<jobid> -Force*
  To kill the job, specify the -Force switch parameter. Note that you can specify multiple job IDs at the same time, separated by commas, but do not support wildcards.
- *Get-OrchJob | Stop-OrchJob*
  Stop-OrchJob accepts piped input from Get-OrchJob. This allows you to stop any running jobs in the current folder. It is convenient because you do not need to specify the job ID.
- *Get-OrchJob -Process <process names> | Stop-OrchJob -WhatIf*
  Stops all jobs for a given process name. If you add -WhatIf, you can check the job ID to be stopped without actually stopping it.
- *Get-OrchJob -Recurse | Stop-OrchJob -Force*
  Kill all running jobs in the current folder and all its subfolders. Stop-OrchJob doesn't have a -Recurse option, but you can pipe the output of Get-OrchJob -Recurse into Stop-OrchJob to stop all running jobs in all subfolders at once.

Open Jobs in the browser

- *Open-OrchJob <job ids>*
  Open specified jobs in the browser.

Job Recordings

- *Get-OrchJobVideo [-Skip <int>] [-First <int>] [-Path <string[]>] [-Recurse] [-Depth <uint>]*

Required permissions: OR.Job.Read

Get Jobs that have Recordings (video) attached.

- *Get-OrchJobMedia [-Skip <ulong>] [-First <ulong>] [-Path <string[]>] [-Recurse] [-Depth <uint>]*

  Required permissions: OR.Monitoring.Read

  Get Jobs that have Recordings (screenshots) attached.
- *Export-OrchJobMedia [[-JobId] <long[]>] [[-Destination] <string>] [-Path <string[]>] [-Recurse] [-Depth <uint>] [-WhatIf] [-Confirm]*

  Required permissions: OR.Monitoring.Read

  Downloads the screenshots attached to the job and saves them to a file. The -Destination parameter specifies the local folder where the files will be saved.
- *Remove-OrchJobMedia [-JobId] <string[]> [-Path <string[]>] [-Recurse] [-Depth <uint>] [-WhatIf] [-Confirm]*

  Required permissions: OR. Monitoring.Write

  Remove Job Recordings (screenshots) from jobs. Specify the -JobId for which you want to remove the job recording in the first parameter. The -JobId can be specified as multiple comma-separated texts, including wildcards. Autocomplete input for the JobId will display candidates that have been retrieved by *Get-OrchJobMedia*.

Get Jobs that have video attached

- *Get-OrchJobVideo*

  Get all jobs with video attached in the current folder. To view the videos of the retrieved jobs, execute *Open-OrchJob <JobID>* to open these jobs in a browser.
- *Get-OrchJobVideo -Recurse*

  Retrieves all jobs with video attached in the current folder and its subfolders. When executed in the root folder, it retrieves all jobs with videos attached in this tenant.

Get Jobs that have screenshots attached

- *Get-OrchJobMedia*

  Retrieves all job recordings in the current folder.
- *Get-OrchJobMedia -Recurse*

  Get all Job Recordings in the current folder and all its subfolders. When run in the root folder, it retrieves all job recordings for this tenant.
- *Get-OrchJobMedia -Skip 3 -First 5*

For the job recordings in the current folder, skips the first 3 entries and retrieves the next 5 entries.

Save screenshots to files

- *Export-OrchJobMedia*
  Saves all screenshots of jobs stored in the current folder to a file. If no parameter is specified, the files will be saved to the current location on the C: drive. The file names will be as follows:
  fid<folderId>_<processName>_<jobId>_ScreenCapture.zip
  To open the current location on the C: drive in File Explorer, execute *ii c:*.
- *Export-OrchJobMedia -Recurse*
  Saves all screenshots of jobs stored in the current folder and its subfolders to files. When executed in the root folder, it saves all screenshots stored on this drive (tenant) to files.
- *Export-OrchJobMedia -Destination c:¥temp*
  Saves all screenshots of jobs stored in the current folder to c:¥temp.

Remove screenshots from Jobs

- *Remove-OrchJobMedia <JobId>*
  Remove the Job Recordings for the specified JobId from the current folder. The JobId can be specified as multiple comma-separated texts, including wildcards. Autocomplete input for the JobId will display candidates that have been retrieved by *Get-OrchJobMedia*.
- *Remove-OrchJobMedia <jobId> -WhatIf*
  Remove the Job Recording for the specified JobId from the current folder. The -WhatIf parameter indicates to confirm which Job Recordings would be removed without actually performing the removal.
- *Remove-OrchJobMedia <jobId> -Recurse*
  Remove the Job Recording for the specified JobId from the current folder and all its subfolders.

Logs

- *Get-OrchLog [-Id] <long[]> [[-Level] <string>] [-State <string[]>] [-TimeStampAfter <datetime>] [-Skip <ulong>] [-First <ulong>] [-Path < string[]>]*
  Required permissions: OR. Monitoring.Read
  Get the execution logs of the Robot.

Get logs

- *Get-OrchLog <job ids>*
  The job ID (the value of the -Id parameter) can be supplemented with the job ID already obtained by Get-OrchJob. You can specify multiple job IDs separated by commas.
- *Get-OrchLog <job ids> -level <log level>*
  Outputs only lines that are above or equal to the specified log level. The log level can be completion. The default value for -Level is Info.
- *Get-OrchLog <job ids> -Skip 100 -First 20*
  Skip the first 100 lines and print only the next 20 lines. It can be used in conjunction with other parameters, such as -Level.
- *Get-OrchLog<job ids> -TimeStampAfter '2024/01/11 0:00'*
  Get-OrchLog cannot retrieve logs with more than 10,000 lines. If the log expires, you can retrieve the continuation with the -TimeStampAfter parameter.
- *Get-OrchLog -State Faulted [Ctrl+Space]*
  The -State parameter narrows down the jobs to list in the job ID completion input. The above example lists only the IDs of the jobs that failed. Note that the -State parameter does not affect the execution result of *Get-OrchLog*.

Audit Logs

Audit logs are a tenant entity. Therefore, the cmdlets in this section work with the audit log of the current drive (tenant). (The current location does not affect the operation)-Path parameter should specify the drive (tenant) to be operated on. Auto-completion feature is available when typing the drive names. You can specify multiple drive names, separated by commas. Note that wildcards are not allowed in drive names. If -Path is not specified, the audit log of the current drive is manipulated.

- *Get-OrchAuditLog [[-Last] <string>] [[-Component] <string[]>] [[-UserName] <string[]>] [[-Action] <string[]>] [-ExecutionTimeAfter <datetime>] [-ExecutionTimeBefore <datetime>] [-ExpandEntity] [-Skip <ulong>] [-First <ulong>] [-Path <string[]>] [<CommonParameters>]*
- *Get-OrchAuditLog [-Id <string[]>] [-ExpandEntity] [-Skip <ulong>] [-First <ulong>] [-Path <string[]>] [<CommonParameters>]*
  Required permissions: OR. Audit.Read
  Gets the audit logs. In the first set of parameters, you can specify the conditions for the audit logs that you want to retrieve with different parameters. In the second parameter

set, you can directly specify the ID of the audit log to be retrieved in the -Id parameter. The completion list for this ID will only display the Ids of the (cached) audit logs obtained in the first set of parameters. For this reason, first of all, after getting multiple jobs with the first parameter set, if there is a job that you want to check in detail, specify its Id as the -Id parameter and re-execute it.
-ExpandEntity expands the details of the audit log.

Get Audit Logs

- *Get-OrchAuditLog*
  View all audit logs. Audit logs are a tenant entity, so you can run them under any folder to get the same results.
- *Get-OrchAuditLog Day*
  Show the audit log output within the last 24 hours.
- *Get-OrchAuditLog -Component Asset -ExpandEntity*
  View the audit log for the ASSET. The -ExpandEntity parameter instructs you to expand the entity rows that are included in the audit log.
- *Get-OrchAuditLog -Id <audit log id> -ExpandEntity*
  Show the audit log for the specified ID. The audit log ID completion candidate shows the ID cached by *the previous Get-OrchAuditLog*.
  You can also use parameters such as -UserName and -Action to filter the audit logs that are displayed.

Robots

A robot is a tenant entity. Therefore, the cmdlets in this section operate on the robot of the current drive (tenant). (The current location does not affect the operation)-Path parameter should specify the drive (tenant) to be operated on. Auto-completion feature is available when typing the drive names. You can specify multiple drive names, separated by commas. Note that wildcards are not allowed in drive names. If -Path is not specified, the robot on the current drive is operated.

- *Get-OrchRobot [[-FullName] <string[]>] [-Path <string[]>]*
  Required permissions: OR. Robots.Read
  Get a robot.

Get All Robots in a Tenant

- *Get-OrchRobot*
  Show all robots in the current drive (tenant).
- *Get-OrchRobot -Path Orch1:,Orch2:*
  Show all robots in the specified drive (tenant). The -Path parameter allows you to provide complete drive names.

Get Robots with a Specified Condition

- *Get-OrchRobot <robot name>*
  Show all robots with the specified name in the current drive (tenant). Auto-completion feature is available when typing the robot names. You can specify multiple robot names with wild cards, separated by commas.
- *Get-OrchRobot | ? {$_. Type -eq "StudioPro"}*
  View all Robots of type StudioPro.

Users and Groups

Users and groups are tenant entities. Therefore, the cmdlets in this section work with users and groups on the current drive (tenant). (The current location does not affect the operation) -Path parameter specifies the drive (tenant) to be operated on. Auto-completion feature is available when typing the drive names. You can specify multiple drive names separated by commas. Note that wildcards are not allowed in drive names. If -Path is not specified, the current drive is operated.

- *Get-OrchUser [[-FullName] <string[]>] [[-UserName] <string[]>] [-Path <string[]>]*
  Required permissions: OR. Users.Read
  Gets a user. In the first parameter, specify multiple FullNames of the users you want to retrieve, separated by commas and wildcards. If FullName is omitted, all users are retrieved. You can also specify a UserName with the -UserName parameter.

- *Remove-OrchUser [[-FullName] <string[]>] [[-UserName] <string[]>] [-Path <string[]>] [-WhatIf] [-Confirm]*
  Required permissions: OR. Users.Write

Remove a user. The user can be specified with -FullName or -UserName.

- *Get-OrchCurrentUser [-Path <string[]>]*
  Required permissions: OR. User.Read
  Show the users who are connected to the specified drive. Please use it for a drive connected with a non-confidential application (user scope). It does not work for drives connected with confidential apps (app scope).

- *Update-OrchCurrentUserURPassword [-Path <string[]>] [-WhatIf] [-Confirm]*
  Required permissions: OR. Users
  Updates the password of the Unattended Robot for the current user. Immediately after you run this cmdlet, you are prompted to enter your password interactively.

View the users in tenants

- *Get-OrchUser*
  View all users in this drive (tenant).
- *Get-OrchUser -Path Orch1:,Orch2:*
  Show all users on the drive (tenant) specified by the -Path parameter.
- *Get-OrchUser a*,y**
  Show all users in this drive (tenant) that start with A and users that start with Y. Search by the user's FullName. The value specified for -FullName can be auto-completed by pressing the [Tab] or [Ctrl+Space] keys.
- *Get-OrchUser -UserName a*,y**
  Show all users in this drive (tenant) that start with A and users that start with Y. If specified by the -UserName parameter, the search is performed by the user's UserName. The value specified for -UserName can also be auto-completed.
- *Get-OrchUser | ? { $_.RolesList -contains 'Administrator' }*
  Displays only users with the Administrator role.

Remove users from tenants

- *Remove-OrchUser <user names>*
  Remove a user with the specified username from this drive (tenant). This username is FullName. You can use wildcards in user names. You can also specify multiple user names, separated by commas. Completion input is also available.
- *Remove-OrchUser -UserName <user names>*

Remove a user with the specified username from this drive (tenant). If the -UserName parameter name is specified, search by UserName. You can use wildcards in your user name. You can also specify multiple user names, separated by commas. Completion input is also available.

- *Remove-OrchUser -Path Orch1: <user names>*
  You can also specify a drive names. You can specify multiple drive names separated by commas, but wildcards are not allowed.

Confirm the current user

The *Get-OrchCurrentUser* cmdlet is available for non-confidential apps (user-scoped) connected drives. Please note that it does not work with drives connected with confidential apps (app scope).

- *Get-OrchCurrentUser*
  Show the users logged in with UiPathOrch in the current drive (tenant). Note that this user may be different for each drive.
- *Get-OrchCurrentUser -Path Orch1:,Orch2:*
  Show the users logged in with UiPathOrch in a given drive (tenant). The -Path parameter accepts multiple drives separated by commas. Completion input is also available.

Update the Unattended Robot's password for the current user

The *Update-OrchCurrentUserURPassword* cmdlet is available for non-confidential apps (user-scoped) connected drives. Please note that it does not work with drives connected with confidential apps (app scope). As a result, you also cannot change the unattended robot password for a robot account. You can only change the Unattended Robot password for a user account.

- *Update-OrchCurrentUserURPassword*
  Run it without any parameters. Interactively, prompts for a password. The password is updated when the current user's Unattended Robot logs in to Windows.
- *Update-OrchCurrentUserURPassword -Path Orch1:,Orch2:*
  The -Path parameter allows you to specify the target drive (tenant). If you specify multiple drives, update the password of the Unattended Robots of the current user connected to them to the same password at once. With the -WhatIf parameter, you can see the current user on each drive.

Roles

A role is a tenant entity. Therefore, the cmdlets in this section work with the role of the current drive (tenant). (The current location does not affect the operation)-Path parameter should specify the drive (tenant) to be operated on. Auto-completion feature is available when typing the drive names. You can specify multiple drive names, separated by commas. Note that wildcards are not allowed in drive names. If -Path is not specified, the role of the current drive is operated.

- *Get-OrchRole [[-DisplayName] <string[]>] [-Path <string[]>] [-ExpandPermission]*
  Required permissions: OR. Users.Read
  Get a role. The switch parameter -ExpandPermission indicates that the permissions contained in the role should be expanded and displayed.

- *Remove-OrchRole [[-DisplayName] <string[]>] [-Path <string[]>] [-WhatIf] [-Confirm]*
  Required permissions: OR. Users.Write
  Remove a role.

- *Copy-OrchRole [-DisplayName] <string[]> [-Destination] <string[]> [-Path <string>] [-WhatIf] [-Confirm]*
  Required permission: OR.Users.Read on the source tenant
  OR.Users.Write in the destination tenant
  Copy roles from one tenant to another.

View Roles

- *Get-OrchRole*
  Show all roles for the current drive (tenant).
- *Get-OrchRole <role names>*
  Show all roles with the specified name in the current drive (tenant). You can specify multiple role names, separated by commas, including wildcards. Completion input is also available.
- *Get-OrchRole <role names> -ExpandPermission*
  The -ExpandPermission parameter expands and displays the details of the permissions included in this role.

Remove Roles

- *Remove-OrchRole <role names>*
  Removes the role with the specified name from the tenant. You can specify multiple names that contain wildcards, separated by commas. Completion input is also available.
- *Remove-OrchRole -Path Orch1:,Orch2 a**
  Remove all roles that begin with a in all of the specified drives (tenants).
- *Remove-OrchRole -Path Orch1:,Orch2 a* -WhatIf*
  Remove all roles that begin with a in all of the specified drives (tenants). The -WhatIf parameter indicates that you should not actually remove it, but rather check the name of the role that you want to remove. If you specify -Confirm instead of -WhatIf, a confirmation message is displayed each time a role is removed.

Copying roles between tenants

- *Copy-OrchRole * Orch2:*
  Copy all roles of the current drive (tenant) to Orch2:.
- *Copy-OrchRole * Orch2: -WhatIf*
  Copy all roles of the current drive (tenant) to Orch2:. The -WhatIf parameter indicates that you don't actually want to copy it, but to check the name of the role that you want to copy. If you specify -Confirm instead of -WhatIf, you will see a confirmation message each time you copy a role.
- *Copy-OrchRole -Path Orch2: a* .*
  Copy all the roles starting with a in the Orch2: drive (tenant) to the tenant of the current drive.
- *Copy-OrchRole * Orch2:,Orch3:*
  Copy all roles for the current drive (tenant) to both Orch2: and Orch3:.

Folder Users

The cmdlets in this section can use either -FullName, -UserName, or both to specify the user to work with. I think -FullName is easier to use, but keep in mind that -FullName may not be unique. -FullName is a positional parameter, so you do not need to specify a parameter name. When specifying a user with -UserName, specify it together with the parameter name of -UserName. You can use wildcards for both FullName and UserName. The -WhatIf parameter is useful to see who you want to work with. You can also use wildcards in roles.

- *Get-OrchFolderUser [[-FullName] <string[]>] [[-UserName] <string[]>] [-IncludeInherited] [-Path <string[]>] [-Recurse] [-Depth <uint>]*
  Required permissions: OR. Folders.Read
  Gets the users assigned to the folder.

- *Add-OrchFolderUser [[-FullName] <string[]>] [-Roles] <string[]> [-UserName <string[]>] [-Path <string[]>] [-WhatIf] [-Confirm]*
  Required permissions: OR. Folders
  Assigns a user to a folder.

- *Remove-OrchFolderUser [[-FullName] <string[]>] [[-UserName] <string[]>] [-Path <string[]>] [-Recurse] [-Depth <uint>] [-WhatIf] [-Confirm]*
  Required permissions: OR. Folders
  Removes a user from a folder.

- *Add-OrchRoleToFolderUser [[-FullName] <string[]>] [-Roles] <string[]> [-UserName <string[]>] [-Path <string[]>] [-Recurse] [-Depth <uint>] [-WhatIf] [-Confirm]*
  Required permissions: OR. Folders
  Add a role to a folder user.

- *Remove-OrchRoleFromFolderUser [[-FullName] <string[]>] [-Roles] <string[]> [-UserName <string[]>] [-Path <string[]>] [-Recurse] [-Depth <uint>] [-WhatIf] [-Confirm]*
  Required permissions: OR. Folders
  Removes a role from a folder user.

View Users Assigned to a Folder

- *Get-OrchFolderUser*
  Show all the users assigned to the current folder. It does not show users that have inherited from the parent folder.
- *Get-OrchFolderUser -IncludeInherited*
  It also shows the users that inherited from the parent folder.
- *Get-OrchFolderUser -Recurse*

Show all the users assigned to the current folder and its subfolders. When run in the root folder, it displays the users in all folders.

- *Get-OrchFolderUser - IncludeInherited*
  Show all the users assigned to the current folder. It also shows the users that inherited from the parent folder.
- *Get-OrchFolderUser *yoshi**
  Show all users assigned to the current folder that contain yoshi in FullName. The user name can be multiple comma-separated names, including wildcards. Completion input is also available. If you want to search by UserName, specify it with the -UserName parameter.
- *Get-OrchFolderUser -Path <folder names>*
  Show the users assigned to a specific folder. If you specify a path that includes a drive names (for example, Orch1:¥shared), you can also run it from a drive from a different provider. The folder name can be multiple comma-separated names, including wildcards.

Remove Users from folders

- *Remove-OrchFolderUser username*
  Removes the user assigned to the current folder from the folder. The user name is searched for in FullName. The user name can be multiple comma-separated names, including wildcards.
- *Remove-OrchFolderUser -Recurse a**
  Removes users whose FullName starts with a from the current folder and all of its subfolders.
- *Remove-OrchFolderUser -Recurse -UserName a* -WhatIf*
  Removes users whose UserName starts with a from the current folder and all of its subfolders. The -WhatIf parameter indicates that you don't actually want to remove them, but that you want to see which users you want to remove.
- *Remove-OrchFolderUser -Recurse -UserName a* -Confirm*
  Removes users whose UserName starts with a from the current folder and all of its subfolders. The -confirm parameter displays a confirmation message each time a user is removed.

Assign users to folders

- *Add-OrchRoleToFolderUser <user names> <role names>*
  Assigns the user to the current folder. The user name is specified by FullName. If you

want to specify it by UserName, specify it with the -UserName parameter. You can specify multiple user names, separated by commas, including wildcards. Completion input is also available. This completion input only displays the users that are already assigned to the folder.
At least one role must be specified. You can also specify multiple roles with wildcards, separated by commas. Completion input is also available.

- *Add-OrchRoleToFolderUser <user names> <role names> -WhatIf*
  Similar to the above, but without actually adding it, we will see which users we want to add.

Remove roles from a user who has already been assigned to a folder

- *Remove-OrchRoleFromFolderUser <user names> <role names>*
  You can specify multiple user names and role names, each separated by commas, including wildcards. For completion input, only the users and roles that are already assigned to the current folder are displayed.

Machines

A machine is a tenant entity. Therefore, the cmdlets in this section work with machines on the current drive (tenant). (The current location does not affect the operation)-Path parameter should specify the drive (tenant) to be operated on.

- *Get-OrchMachine [[-Name] <string[]>] [-Path <string[]>] [-ExpandRobotUser]*
  Required permissions: OR. Machines.Read
  Get a machine. -ExpandRobotUser displays the account-to-robot mappings for each machine.

- *Remove-OrchMachine [-Name] <string[]> [-Path <string[]>] [-WhatIf] [-Confirm]*
  Required permissions: OR. Machines.Write
  Remove the machine.

- *Copy-OrchMachine [-Name] <string[]> [-Destination] <string[]> [-Path <string>] [-WhatIf] [-Confirm]*
  Required permissions: OR.Machines.Read on the source tenant
  OR.Machines.Write for the destination tenant
  Copy the machines between tenants.

View tenant's machines

- *Get-OrchMachine*
  View all machines on this drive (tenant).
- *Get-OrchMachine <machine names>*
  Show all machines with the specified name in this drive (tenant). The machine name can contain multiple names that contain wildcards, separated by commas. Completion input is also available.
- *Get-OrchMachine -Path Orch1:,Orch2*
  Lists all machines on a given drive (tenant).
- *Get-OrchMachine -ExpandRobotUser*
  View the account-machine mappings for each machine.

Remove machines in tenants

- *Remove-OrchMachine <machine names>*
  Remove all machines with the specified name on this drive (tenant). The machine name can contain multiple names that contain wildcards, separated by commas. Completion input is also available.

Copy machines between tenants

Please note that in environments where standard machines cannot be created, this cmdlet cannot copy standard machines.

- *Copy-OrchMachine <machine names> Orch2:*
  Copies the specified machine on the current drive to the Orch2: drive. The machine name can contain wildcards, separated by multiple commas. It can also be complemented. You can specify multiple drives for the destination drive (-Destination parameter).
- *Copy-OrchMachine <machine names> Orch2: -WhatIf*
  Instead of actually copying it, it checks the machine name to be copied. If you specify -Confirm instead of -WhatIf, you will see a confirmation message each time you copy a machine.

Folder Machines

- *Get-OrchFolderMachine [[-Name] <string[]>] [-Path <string[]>] [-Recurse] [-Depth <uint>]*

Required permissions: OR. Folders.Read
Gets the machines assigned to a folder.

- *Add-OrchFolderMachine [-Name] <string[]> [-Path <string[]>] [-Recurse] [-Depth <uint>] [-WhatIf] [-Confirm]*
  Required permissions: OR. Folders.Write
  Assign the machine to a folder.

- *Remove-OrchFolderMachine [-Name] <string[]> [-Path <string[]>] [-Recurse] [-Depth <uint>] [-WhatIf] [-Confirm]*
  Required permissions: OR. Folders.Write
  Removes the machine assigned to the folder from the folder.

Get Folder Machines

- *Get-OrchFolderMachine*
  Show all the machines assigned to the current folder.
- *Get-OrchFolderMachine -Recurse*
  Show all machines assigned to the current folder and all its subfolders. When run in the root folder, it displays the machines that are assigned to all folders.
- *Get-OrchFolderMachine -Recurse <machine names>*
  Searches the current folder and its subfolders for the folder to which the specified machine is assigned. You can specify multiple machine names, separated by commas, including wildcards.

Add Machines to folders

- *Add-OrchFolderMachine <machine names>*
  Assigns the specified machine to the current folder. You can specify multiple machine names, separated by commas, including wildcards.
- *Add-OrchFolderMachine <machine names> -WhatIf*
  The -WhatIf parameter doesn't actually add it, it checks for the machines to be added.

Remove Folder Machines

- *Remove-OrchFolderMachine <machine names>*

Removes a given machine from the current folder. You can specify multiple machine names, separated by commas, including wildcards.

- *Remove-OrchFolderMachine <machine names> -WhatIf*
  The -WhatIf parameter does not actually remove the machine, it checks for the machine to be removed.

Assets

- *Get-OrchAsset [[-Name] <string[]>] [-ValueType <string[]>] [-ExpandUserValue] [-Path <string[]>] [-Recurse] [-Depth <uint>]*
- *Get-OrchAsset -ExportCsv [[-Encoding] <Encoding>] [[-CsvPath] <string>] [-Path <string[]>] [-Recurse] [-Depth <uint>]*
- *Get-OrchAsset -ExportCredentialCsv [[-Encoding] <Encoding>] [[-CsvPath] <string>] [-Path <string[]>] [-Recurse] [-Depth <uint>]*
  Required permissions: OR. Assets.Read
  Gets the asset. The first set of parameters retrieves the assets specified by -Name and -ValueType. The switch parameter -ExpandUserValue indicates that the values per user machine should be expanded and displayed.
  The parameter -ExportCsv in the second parameter set outputs text, boolean, and integer assets to a CSV file.
  The parameter -ExportCredentialCsv in the third parameter set outputs the credential assets to a CSV file.
  The CSV file output by this *cmdlet can* be imported as it is with the *Set-OrchAsset and Set-OrchCredentialAsset cmdlets.*

- *Set-OrchAsset [[-ValueType] <string>] [-Name] <string[]> [[-Value] <string>] [[-UserName] <string[]>] [[-MachineName] <string[]>] [-Description <string>] [-Path <string[]>] [-WhatIf] [-Confirm]*
- *Set-OrchAsset -ExportTemplateCsv [[-Encoding] <Encoding>] [[-TemplateCsvPath] <string>] [-WhatIf] [-Confirm]*
  Required permissions: OR. Assets
  The first set of parameters creates, updates, and removes assets. If you specify an asset name that does not exist, a new asset is created. Providing an existing asset name updates this asset. In this case, if you specify '' (empty string) for the -value parameter, this asset value will be removed.
  The parameter -ExportTemplateCsv in the second parameter set outputs a template in a

CSV file that can be imported with this *Set-OrchAsset* cmdlet.

- *Remove-OrchAsset [-Name] <string[]> [[-ValueType] <string[]>] [-Path <string[]>] [-Recurse] [-Depth <uint>] [-WhatIf] [-Confirm]*
  Required permissions: OR. Assets.Write
  Remove an asset. *Set-OrchAsset* and *Set-OrchCredentialAsset* can remove the specified per-user machine asset value, while *Remove-OrchAsset* can remove the specified per-user machine asset value with or without the, removes the specified asset itself.

View assets and credential assets

- *Get-OrchAsset*
  Show all assets in the current folder. Note that the password for the credential asset is not displayed. This is a limitation by design of the Orchestrator Web API.
- *Get-OrchAsset <asset names>*
  Show all assets in the current folder. The asset name can be multiple comma-separated names, including wildcards.
- *Get-OrchAsset <asset names> -ExpandUserValue*
  Expands and displays the per-user/per-machine asset values for a given asset in the current folder. If a global value is set, the global value is also printed first.
- *Get-OrchAsset -Recurse*
  Show all assets in the current folder and all its subfolders.
- *Get-OrchAsset -ValueType Text*
  Show all text assets in the current folder. If you perform a completion operation immediately after the above cmdlet, only the name of the text asset will be listed.

Add and Update Assets

- *Set-OrchAsset Text <asset names> Asset Value*
  In the current folder, create an asset of type Text with the specified name and set its global value to 'Asset Value'. If an asset with the same name already exists, update its global value. The asset name can be multiple comma-separated names, including wildcards. If you expand a wildcard and it does not match any of the existing assets, it will be treated as a new asset name. If you fill in an asset value, the current value is displayed.
- *Set-OrchAsset Bool Asset <asset names> Value*
  It works the same way for Bool and Integer assets.

- *Set-OrchAsset Text <asset names> <asset value> <user names>*
  If you specify a user name, you can create and update asset values for each user. User name completion displays only the list of users assigned to the target folder. Note that when you specify a wildcard in the username, the user is searched and expanded for all users in the tenant. This is to ensure that it can be processed faster when importing from a CSV file. (Retrieving folder users is a relatively expensive operation in Orchestrator.)
- *Set-OrchAsset Text <asset name> -UserName <user names> [Ctrl+Space]*
  To edit and update the current value of the asset value per user, specify the user name before the asset value and then perform the completion operation. The current value is displayed.
- *Set-OrchAsset Text <asset names> <asset value> <user names> <machine names>*
  Specifying both the username and the machine name allows you to create and update per-user and per-machine asset values.
- *Set-OrchAsset Text <asset names> -UserName <user names> -MachineName <machine names> [Ctrl+Space]*
  To edit and update the current value of the per-user/per-machine asset value, specify the user name and machine name before the asset value, and then perform the completion input operation.
- *Set-OrchAsset Text <asset names> -Description <description>*
  Adds/updates a description for a given asset in the current folder. Note that this cmdlet does not remove the description of the asset, even if you specify '' for the -Description parameter. This behavior is so that even if you specify an empty string in the Description column of the CSV file to be imported, the existing Description will not be removed.

Remove Assets

- *Remove-OrchAsset <asset names>*
  To permanently remove a specified credential asset, use the *Remove-OrchAsset* cmdlet. You can specify multiple asset names, separated by commas, including wildcards. The -Recurse parameter is available.
- *Set-OrchAsset Text <asset names> ''*
  Specifying '' for the asset value removes the value of this asset. In the case of an asset that contains at least one per-user and per-machine asset value, only the global value is removed, and the existing asset itself is not removed. If you do this for an asset that does not contain a per-user value, the asset will be permanently removed.
- *Set-OrchAsset Bool <asset names> ''*

You can also remove Bool and Integer assets by specifying '' as the asset value. In the example above, Bool is specified for the -ValueType parameter. Note that this parameter is only required when creating a new asset. If you are updating an existing asset, you do not need the -ValueType parameter to work. However, because -ValueType is a positional parameter, if this parameter is omitted, the subsequent parameter must be specified with a name.

- *Set-OrchAsset Text <asset names> '' <user names>*
  Given a user name, you remove the per-user asset value for that username. In this case, the global value and the asset value with the machine name are not removed. The user name can be multiple comma-separated names, including wildcards. It can also be complemented. To remove all per-user assets contained in this asset, specify * for the username.
- *Set-OrchAsset Text <asset names> '' <user names> <machine names>*
  If you specify both the user name and the machine name, the matching per-user and per-machine asset values are removed. The user name and machine name can be multiple comma-separated names, including wildcards. It can also be complemented. To remove all asset values in this asset that have both a user name and a machine name, specify * for both the user name and the machine name.

Export assets to a CSV file

When exporting to a CSV file, you do not need to specify the -ExpandUserValue parameter. Asset values for each user and machine are also output automatically. The CSV file output in this section can be imported as it is with the *Set-OrchAsset* cmdlet. This method will be described below.

- *Get-OrchAsset -ExportCsv -Encoding shift_jis*
  Outputs Text/Bool/Integer assets in the current folder to a CSV file. If the CSV file path is not specified, the CSV file will be output to the current location of the C: drive with the default file name ExportedAssets.csv. You can specify the encoding of the CSV file with the -encoding parameter. In a Japanese environment, it is convenient to output in shift_jis so that you can open it in Excel without garbled characters.
- *Get-OrchAsset -ExportCsv -Encoding shift_jis assets.csv*
  Outputs the specified file name to the current location of the C: drive. To move to the current location of C:, use cd c:. To open the current location of C: in File Explorer, use ii c:. Also, c:asset.csv to open the file assets.csv in the current location of C:. You can also use c:a[Ctrl+Space] to complete the file name.

- *Get-OrchAsset -ExportCsv -Encoding shift_jis data/assets.csv*
  This is an example of specifying a path relative to the current location of the C: drive. You can also use both ¥ or ¥ as a folder separator for the C: drive (FileSystem provider).
- *Get-OrchAsset -ExportCsv -Encoding shift_jis c:¥temp*
  This is an example of specifying the absolute path of a folder on the C: drive. In this case, the default file name ExportedAssets.csv is output to the specified folder.
- *Get-OrchAsset -ExportCsv -Encoding shift_jis c:¥temp*
  This is an example of specifying the absolute path of a file on the C: drive. In this case, the file is output with the specified file name.

Import assets from a CSV file

When Set-OrchAsset has multiple rows with the same Name column (asset name) in the CSV file, Set-OrchAsset combines those rows into a single API call. As such, it also processes CSV files with many per-user values at high speed. Assets with empty Value columns in the CSV file will be removed.

- *Import-Csv C:ExportedAssets.csv -Encoding shift_jis | Set-OrchAsset -WhatIf*
  Execute Set-OrchAsset with the column names in each row of the CSV file as parameters. This CSV file can be created by specifying -ExportCsv to Get-OrchAsset. Note that the destination folder is specified in the Path column of the CSV file. The shift_jis is the same for SJIS.
- *Import-Csv C:ExportedAssets.csv -Encoding shift_jis | Set-OrchAsset -Path . -Whatif*
  To import all assets listed in the CSV file into the current folder, specify the destination folder with the -Path parameter of the cmdlet. The Path column in the CSV file is overwritten with the folder specified by the -Path parameter. In the example above, we want to import all the assets listed in the CSV into the current folder (note that this is not interpreted as relative to the folder specified by the -Path parameter).
- *Import-Csv C:ExportedAssets.csv -Encoding shift_jis | Set-OrchAsset -WhatIf*
  If you want to import the asset to a different drive (tenant) than the exported drive, modify the Path column in the CSV file and replace the drive names with the drive names you want to import to. Alternatively, remove the drive names in the Path column and start with the root folder, and run the above command on the destination drive.

Credential Assets

- *Set-OrchCredentialAsset [-Name] <string[]> [[-UserName] <string[]>] [[-MachineName] <string[]>] [[-CredentialUsername] <string>] [[-*

CredentialPassword] <string>] [-Description <string>] [-CredentialStore <string>] [-Path <string[]>] [-WhatIf] [-Confirm]

- *Set-OrchCredentialAsset -ExportTemplateCsv [[-Encoding] <Encoding>] [[-TemplateCsvPath] <string>] [-WhatIf] [-Confirm]*

  Required permissions: OR. Assets

  The first set of parameters creates, updates, and removes credential assets. If you specify an asset name that does not exist in the -name parameter, a new asset is created. If you specify an existing asset name for the -Name parameter, you update this asset. In this case, if you specify '' (empty string) for the -CredentialPassword parameter, this asset value will be removed.

  The parameter -ExportTemplateCsv in the second parameter set outputs a template in a CSV file that can be imported with this *Set-OrchCredentialAsset* cmdlet.

View Credential Assets

You can view credential assets with the *Get-OrchAsset* cmdlet. See the previous section. Due to a limitation in the Orchestrator API, the password cannot be displayed.

Add and Update Credential Assets

- *Set-OrchCredentialAsset <credential asset names>*

  Creates a credential asset in the current folder. It interactively prompts you for a username and password for your credentials, so enter them from the PowerShell console. These will be set to the global value of the provided asset name. If you already have a credential asset with the same name, update its global value. If an asset name contains a wildcard character and it does not match an existing asset when expanded, create a new credential asset.
- *Set-OrchCredentialAsset <credential asset names> <user names>*

  If you specify a user name (-UserName parameter), create and update a credential user and credential password for that user. Username completion displays a list of users assigned to the target folder. Note that when you specify a wildcard in the username, the user is searched and expanded for all users in the tenant. This is to ensure that it can be processed faster when importing from a CSV file. (Retrieving folder users is a relatively expensive operation in Orchestrator.)
- *Set-OrchCredentialAsset <credential asset names> <user names> <machine names>*

  Specifying both the username and the machine name allows you to create and update per-user and per-machine asset values. It interactively prompts you for a username and

password for your credentials, so enter them from the PowerShell console.

- *Set-OrchCredentialAsset <credential asset names> -CredentialUsername <credential user name> -CredentialPassword <credential password>*

  You can also specify the username and password for the credentials in the parameters. In this case, no interactive input is required. In practice, this parameter is often used as the column name in the CSV to be imported.

Removing a Credential Asset

- *Remove-OrchAsset <asset names>*

  To permanently remove a specified credential asset, use the Remove-OrchAsset cmdlet. You can specify multiple asset names, separated by commas, including wildcards. The -Recurse parameter is available.

- *Set-OrchCredentialAsset <asset names> -CredentialPassword ''*

  Specifying '' for the -CredentialPassword parameter removes the value of this asset. If you perform the above on a credential asset that contains at least one per-user and per-machine asset value, only the global value will be removed, not the asset itself. If you do this for a credential asset that does not contain a per-user value, the asset will be permanently removed.

- *Set-OrchCredentialAsset <asset names> '' <user names>*

  Given a user name, you remove the per-user asset value for that username. In this case, the global value and the asset value with the machine name are not removed. The user name can be multiple comma-separated names, including wildcards. It can also be complemented. To remove all per-user assets contained in this asset, specify * for the username.

- *Set-OrchCredentialAsset Text <asset names> '' <user names> <machine names>*

  If you specify both the user name and the machine name, the matching per-user and per-machine asset values are removed. The user name and machine name can be multiple comma-separated names, including wildcards. It can also be complemented. To remove all asset values in this asset that have both a user name and a machine name, specify * for both the user name and the machine name.

Remove Credential Assets

- *Remove-OrchAsset <asset names>*

  To permanently remove a specified credential asset, use the Remove-OrchAsset cmdlet. You can specify multiple asset names, separated by commas, including wildcards. The -Recurse parameter is available.

Export credential assets to a CSV file

The CSV file output in this section can be imported as it is with the Set-OrchCredentialAsset cmdlet.

- *Get-OrchAsset -ExportCredentialCsv -Encoding shift_jis*
  Outputs the Credential assets in the current folder to a CSV file. The method of specifying the path and encoding of the CSV file is the same as when -ExportCsv is specified, so please refer to the previous section.

Import credential assets from a CSV file

Set-OrchCredentialAsset combines multiple rows with the same Name column (asset name) in the CSV file into a single API call. As such, it also processes CSV files with many per-user values at high speed. Credential assets that have both the CredentialUsername and CredentialPassword columns empty in the CSV file will be removed. Note that when removing per-user credentials with cmdlet parameters (without using a CSV file), simply specify -CredentialPassword '' and the credentials will be removed (you do not need to specify -CredentialUsername '').

- *Import-Csv C:ExportedCredentialAssets.csv -Encoding sjis | Set-OrchCredentialAsset -WhatIf*
  Execute Set-OrchCredentialAsset with the column names in each row of the CSV file as parameters. This CSV file can be created by specifying -ExportCredentialCsv to Get-OrchAsset. Note that the destination folder is specified in the Path column of the CSV file.
- *Import-Csv C:ExportedAssets.csv -Encoding sjis | Set-OrchAsset -Path . -Whatif*
  To import all assets listed in the CSV file into the current folder, specify the destination folder with the -Path parameter of the cmdlet. The Path column in the CSV file is overwritten with the folder specified by the -Path parameter. The above example imports all of the credential assets listed in the CSV into the current folder (note that this is not interpreted as relative to the folder specified by the -Path parameter).
- *Import-Csv C:ExportedCredentialAssets.csv -Encoding sjis | Set-OrchCredentialAsset -WhatIf*
  If you want to import the asset to a different drive (tenant) than the exported drive, modify the Path column in the CSV file and replace the drive names with the drive names you want to import to. Alternatively, in the path of the Path column, remove the drive names from the folder name and make it start from the root folder, and run the above

command on the destination drive.

Credential Stores

A credential store is a tenant entity. Therefore, the cmdlets in this section work with credential stores on the current drive (tenant). (The current location does not affect the operation)- Path parameter should specify the drive (tenant) to be operated on.

- *Get-OrchCredentialStore [[-Name] <string[]>] [-Path <string[]>]*
  Required permissions: OR. Settings.Read
  Retrieve the credential store.

- *Remove-OrchCredentialStore [-Name] <string[]> [-Path <string[]>] [-WhatIf] [-Confirm]*
  Required permissions: OR. Settings.Write
  Remove a credential store.

Get Credential Stores

- *Get-OrchCredentialStore*
  Show all credential stores in the current drive (tenant).
- *Get-OrchCredentialStore <credential store names>*
  Show all credential stores with the given name. You can specify multiple credential store names, separated by commas, including wildcards. It can also be complemented.
- *Get-OrchCredentialStore -Path Orch1:,Orch2: <credential store names>*
  In the -Path parameter, you can specify multiple target tenants (drives) separated by commas. It can also be complemented.

Remove Credential Stores

- *Remove-OrchCredentialStore <credential store names>*
  Remove the credential store with the given name from the current drive (tenant).
- *Remove-OrchCredentialStore -Path Orch1:,Orch2 <credential store names>*
  Remove the credential store with the specified name from the specified drive (tenant).
- *Remove-OrchCredentialStore -Path Orch1:,Orch2 <credential store names> -WhatIf*
  The -WhatIf parameter does not actually remove the credential store, it displays the credential store to be removed.

Queues

- *Get-OrchQueue [[-Name] <string[]>] [-Path <string[]>] [-Recurse] [-Depth <uint>]*
  Required permissions: OR. Queues.Read
  Get a queue.

- *Remove-OrchQueue [[-Name] <string[]>] [-Path <string[]>] [-WhatIf] [-Confirm]*
  Required permissions: OR. Queues.Write
  Remove a queue.

- *Get-OrchQueueItem [-Name] <string[]> [-Skip <ulong>] [-First <ulong>] [-Path <string[]>]*
  Required permissions: OR. Queues.Read
  Retrieves a queue item.

- *Import-OrchQueueItem [-Name] <string[]> [-CsvPath] <string[]> [[-Encoding] <Encoding>] [[-CommitType] <string>] [-Path <string[]>] [-WhatIf] [-Confirm]*
  Required permissions: OR. Queues.Write
  Upload CSV files and bulk add queue items. The -CsvPath parameter allows you to specify either the file name, the folder, or both using wildcards. Note that even if the same file is specified multiple times, it will be uploaded only once.

Get Queues

- *Get-OrchQueue*
  Show all queues in the current folder.
- *Get-OrchQueue <queue names>*
  Show all queues with a given name in the current folder. The queue name can contain multiple names that contain wildcards, separated by commas. It can also be complemented.
- *Get-OrchQueue -Recurse*
  Show all queues in the current folder and its subfolders. When run in the root folder, it displays all queues on this drive (tenant).
- *Get-OrchQueue -Recurse *cue**
  Show all queues in the current folder and its subfolders that contain cue in their name. If you run it in the root folder, you can search for queues on this drive (tenant).

Remove Queues

- *Remove-OrchQueue <queue names>*
  Remove all queues with the given name in the current folder. The queue name can contain multiple names that contain wildcards, separated by commas. It can also be complemented.
- *Remove-OrchQueue -Recurse <queue names>*
  Remove all queues with the given name in the current folder and its subfolders.
- *Remove-OrchQueue <queue names> -WhatIf*
  The -WhatIf parameter does not actually remove, it checks for the queue name to be removed.

Get Queue Items

- *Get-OrchQueueItem <queue names>*
  Retrieves the items in the current folder for the specified queue. The items are displayed in ascending order by Id.
- *Get-OrchQueueItem <queue names> | ? {$_. Status -eq "New "}*
  Retrieves only the items in the specified queue that have the status New.
- *Get-OrchQueueItem <queue names> -skip 5 -first 10*
  Skips the first 5 items in a given queue and retrieves the next 10.

Upload CSV files and bulk add queue items

- *Import-OrchQueueItem <queue names> c:¥temp¥items.csv*
  Uploads the specified CSV file to the specified queue.
- *Import-OrchQueueItem <queue names> c:¥temp¥test*.csv -Encoding shift_jis*
  Allows you to specify multiple CSV files at once using wildcards. Additionally, the -Encoding parameter lets you specify the encoding of the CSV files. (On the Orchestrator web interface, only CSV files in UTF8 can be uploaded.)
- *Import-OrchQueueItem <queue names> c:¥temp*
  If you specify a folder path with the -CsvPath parameter, it uploads all files with a .csv extension contained in that folder.
- *Import-OrchQueueItem <queue names> c:¥temp¥items.csv shift_jis AllOrNothing*
  The -CommitType parameter allows you to specify the strategy for registration. You can specify one of the following: AllOrNothing, StopOnFirstFailure, or

ProcessAllIndependently. If the -CommitType parameter is not specified, ProcessAllIndependently is used as the default value.

Triggers

The cmdlets in this section work with time triggers and queue triggers. Note that it does not support event triggers and API triggers.

- *Get-OrchTrigger [[-Name] <string[]>] [-Path <string[]>] [-Recurse] [-Depth <uint>]*
  Required permissions: OR. Jobs.Read
  Get the trigger.

- *Remove-OrchTrigger [-Name] <string[]> [-Path <string[]>] [-Recurse] [-Depth <uint>] [-WhatIf] [-Confirm]*
  Required permissions: OR. Jobs.Write
  Remove the trigger.

- *Enable-OrchTrigger [-Name] <string[]> [-Path <string[]>] [-Recurse] [-Depth <uint>] [-WhatIf] [-Confirm]*
  Required permissions: OR. Jobs.Write
  Enable the trigger.

- *Disable-OrchTrigger [-Name] <string[]> [-Path <string[]>] [-Recurse] [-Depth <uint>] [-WhatIf] [-Confirm]*
  Required permissions: OR. Jobs.Write
  Disable the trigger.

Show Triggers

- *Get-OrchTrigger*
  Show all the triggers in the current folder.
- *Get-OrchTrigger <trigger names>*
  Show the trigger with the given name in the current folder. You can specify multiple trigger names, separated by commas, including wildcards. It can also be complemented.
- *Get-OrchTrigger -Recurse*
  Show all the triggers in the current folder and its subfolders. When run in the root folder, it displays all triggers for this tenant.

- *Get-OrchTrigger -Recurse | ? {$_.Enabled -eq $true}*
  Show all enabled triggers.
- *Get-OrchTrigger -Recurse | select Path,Name,StartProcessCron*
  Show the Cron expression for each trigger.
- *Get-OrchTrigger | ? QueueDefinitionId -eq 66331 | select PackageName*
  Show the package name for queue triggers where QueueDefinitionId is 66331. Note that this simplified syntax was introduced in PowerShell 3.0.
  https://learn.microsoft.com/powershell/module/microsoft.powershell.core/about/about_simplified_syntax

Remove Triggers

- *Remove-OrchTrigger <trigger names>*
  Remove the trigger with the given name in the current folder. You can specify multiple trigger names, separated by commas, including wildcards. It can also be complemented. The trigger name cannot be omitted. * removes all triggers in the current folder.
- *Remove-OrchTrigger <trigger names> -confirm*
  Remove the trigger with the given name in the current folder. A confirmation message is displayed on each trigger deletion.

Enable Triggers

- *Enable-OrchTrigger <trigger names>*
  Enables the trigger with the given name in the current folder. You can specify multiple trigger names, separated by commas, including wildcards. It can also be complemented. The trigger name cannot be omitted. * enables all triggers in the current folder.
- *Enable-OrchTrigger <trigger names> -WhatIf*
  Enables the trigger with the given name in the current folder. -WhatIf doesn't actually enable it, it checks for the trigger you want to enable.

Disable Triggers

- *Disable-OrchTrigger <trigger names>*
  Disables the trigger with the given name in the current folder. You can specify multiple trigger names, separated by commas, including wildcards. It can also be complemented. The trigger name cannot be omitted. * disables all triggers in the current folder.
- *Disable-OrchTrigger -Recurse <trigger names>*

Disables the trigger with the given name in the current folder and its subfolders. When run in the root folder, it disables any trigger with the specified name on this tenant.

Tests

- *Get-OrchTestCase [[-Name] <string[]>] [-Path <string[]>] [-Recurse] [-Depth <uint>]*
  Required permissions: OR. TestSets.Read
  Get the test case.

- *Get-OrchTestSet [[-Name] <string[]>] [-Path <string[]>] [-Recurse] [-Depth <uint>]*
  Required permissions: OR. TestSets.Read
  Get the test set.

- *Get-OrchTestSetExecution [[-Name] <string[]>] [-Path <string[]>] [-Recurse] [-Depth <uint>]*
  Required permissions: OR. TestSetExecutions.Read
  Get test executions.

View Test Cases

- *Get-OrchTestCase*
  View all test cases in the current folder.
- *Get-OrchTestCase -Recurse*
  View all test cases in the current folder and its subfolders.
- *Get-OrchTestCase -Recurse -Path Orch1:¥*
  View all test cases in the Orch1: drive.
- *Get-OrchTestCase <testcase names>*
  Show all the test cases with the given name in the current folder. You can specify multiple test case names, separated by commas, including wildcards. It can also be complemented.

View Test Sets

- *Get-OrchTestSet*

Show all test sets in the current folder.

- *Get-OrchTestSet -Recurse*
  View all test sets in the current folder and its subfolders.
- *Get-OrchTestSet -Recurse -Path Orch1:¥*
  View all test sets in the Orch1: drive.
- *Get-OrchTestSet <testset names>*
  Show all test sets with the given name in the current folder. You can specify multiple test set names, separated by commas, including wildcards. It can also be complemented.

View Test Executions

- *Get-OrchTestExecution*
  View all test executions in the current folder.
- *Get-OrchTestExecution -Recurse*
  View all test executions in the current folder and its subfolders.
- *Get-OrchTestExecution -Recurse | ? Status -eq Failed*
  View all failed test executions in the current folder and its subfolders.
- *Get-OrchTestExecution -Recurse -Path Orch1:¥*
  View all test executions in the Orch1: drive.
- *Get-OrchTestExecution <testexecution names>*
  Show all the test executions with the given name in the current folder. You can specify multiple test execution names, separated by commas, including wildcards. It can also be complemented.

Starts Test Set

- *Start-OrchTestSet <testset names>*
  Starts the specified test sets in the current folder. You can specify multiple test set names separated by commas, including names with wildcards.
- *Start-OrchTestSet * -WhatIf*
  Starts all test sets in the current folder. The -WhatIf parameter instructs to display the test sets that would be executed without actually running them.
- *Start-OrchTestSet * -Confirm*
  Starts all test sets in the current folder. The -Confirm parameter asks for confirmation before executing each test set.
- *Start-OrchTestSet * -Recurse -Verbose*
  Starts all test sets in the current folder and its subfolders. The -Verbose parameter

displays the test sets being executed in the console.

Stops Test Set Executions

- *Stop-OrchTestExecution <test set execution ids>*
  Stops the execution of the test set with the specified Id in the current folder. Wildcards cannot be used for this Id, but multiple Ids can be specified simultaneously using comma separation. Autocomplete input is also supported.

Alerts

Alerts are a tenant entity. Therefore, the cmdlets in this section work with alerts on the current drive (tenant). (The current location does not affect the operation)-Path parameter should specify the drive (tenant) to be operated on.

- *Get-OrchAlert [[-Last] <string>] [[-Severity] <string>] [[-Component] <string[]>] [-CreationTimeAfter <datetime>] [-CreationTimeBefore <datetime>] [-Skip <ulong>] [-First <ulong>] [-Path <string[]>]*
  Required permissions: OR. Monitoring.Read
  Get alerts.

View Alerts

- *Get-OrchAlert*
  View all alerts.
- *Get-OrchAlert -Last Day*
  Show all alerts from the last 24 hours. In addition, you can specify multiple parameters at the same time to filter the alerts you want to view.

Miscellaneous operations

- *Clear-OrchCache [[-Path] <string[]>]*
  Required permissions: None
  Clear the in-memory cache of UiPathOrch. Clear the in-memory cache with this cmdlet if you want the UiPathOrch console to reflect updates to entities on Orchestrator (such as creating or removing folders) made via Orchestrator Web or external tools. For the -Path parameter, specify the drive on which you want to clear the cache. If the -Path parameter is not specified, the cache on the current drive is cleared. If the current drive

is not an Orch drive, clear the cache on all Orch drives.

- *Edit-OrchConfig [[-Editor] <EditConfigCommand+EditorType>]*
  Required permissions: None
  Open the UiPathOrch configuration file. -Editor can be Default or Notepad. If not specified, the . Open the configuration file in the default text editor associated with the JSON extension. The path to the configuration file is %UserProfile%¥Documents¥PowerShell¥Modules¥UiPathOrch¥UiPathOrchConfig.json.